



单位代码 10006

学 号 22373311

分 类 号 TP311

北京航空航天大学
BEIHANG UNIVERSITY

毕业设计 (论文)

基于 EDA 工具反馈的 LLM Verilog
生成与自动优化

学 院 名 称 计算机学院

专 业 名 称 计算机科学与技术

学 生 姓 名 张金涛

指 导 教 师 王锐

2026 年 5 月

北京航空航天大学

本科生毕业设计（论文）任务书

I、毕业设计（论文）题目：

基于 EDA 工具反馈的 LLM Verilog 生成与自动优化

II、毕业设计（论文）使用的原始资料（数据）及设计技术要求：

1. AutoChip 原论文

2. VerilogEval、RTLLM 评估基准

3. Icarus Verilog 开源 EDA 工具

4. 大语言模型 API（Claude、GPT 系列）

5. 北航本科毕业设计相关规范

III、毕业设计（论文）工作内容：

1. 设计并实现基于 EDA 工具反馈的 Verilog 生成与自动优化系统

2. 接入 VerilogEval 和 RTLLM 双 benchmark

3. 在三个不同能力等级的 LLM 上验证反馈循环有效性

4. 通过消融实验分离反馈信息与多采样的贡献

5. 探索反馈粒度、多轮对话和提示策略的影响

6. 建立实验产物管理与审计机制

IV、主要参考资料：

- [1] Thakur S, et al. AutoChip: Automating HDL Generation, 2023
- [2] Liu M, et al. VerilogEval: Evaluating LLMs for Verilog, ICCAD 2023
- [3] Lu Y, et al. RTLLM: Open-Source Benchmark for RTL Generation, 2024
- [4] Liu S, et al. RTLCoder: LLM-Assisted RTL Code Generation, 2024
- [5] Tsai Y, et al. RTLFixer: Fixing RTL Syntax Errors with LLMs, 2024
- [6] Yao Z, et al. HDLDebugger: Streamlining HDL Debugging, 2025
- [7] Wei J, et al. Chain-of-Thought Prompting, NeurIPS 2022
- [8] Brown T, et al. Language Models are Few-Shot Learners, NeurIPS 2020

____ 计算机 ____ 学院 ____ 计算机科学与技术 ____ 专业类 ____ 220611 ____ 班

学生 ____ 张金涛 ____

毕业设计(论文)时间： ____ 2025 ____ 年 ____ 12 ____ 月 ____ 29 ____ 日至 ____ 2026 ____ 年 ____ 5 ____ 月 ____ 22 ____ 日

答辩时间： ____ 2026 ____ 年 ____ 6 ____ 月 ____ 3 ____ 日

成 绩： ____

指导教师： ____

兼职教师或答疑教师（并指出所负责部分）：

____ 系（教研室）主任（签字）： ____

注：任务书应该附在已完成的毕业设计（论文）的首页。



本人声明

我声明，本论文及其研究工作是由本人在导师指导下独立完成的，在完成论文时所利用的一切资料均已在参考文献中列出。

作者：张金涛

签字：张金涛

时间：2026年5月



基于 EDA 工具反馈的 LLM Verilog 生成与自动优化

学 生：张金涛

指导教师：王锐

摘 要

当前大语言模型已经能够根据自然语言描述生成 Verilog 代码，但在零样本条件下，生成结果在涉及复杂状态机、精确位宽操作和算法型 RTL 设计时仍容易出现编译错误或仿真不匹配。本文关注的问题是：能否利用 EDA 工具已经给出的编译和仿真结果，引导模型对错误代码进行有针对性的修复，从而提升 RTL 代码的自动生成质量。

围绕这一问题，本文构建了一个基于 EDA 工具反馈的 Verilog 生成与自动优化系统。系统以 Icarus Verilog 作为编译仿真后端，将编译器和仿真器的输出解析为结构化反馈信号，引导大语言模型在“生成—编译—仿真—反馈—修复”的反馈循环中定向修复代码错误。系统支持 VerilogEval-Human 与 RTLLM 双 benchmark 接入，可通过统一 API 网关切换不同模型，并提供五级反馈粒度和四种提示策略的灵活配置。

在三个不同能力等级的模型上执行了七组递进式对照实验。在 VerilogEval-Human 20 题子集上，Haiku 的通过率从 80% 提升至 90%；在难度更高的 RTLLM_STUDY_12 上，GPT-5.4 从 50% 提升至 83%。消融实验将反馈收益分解为两个来源：错误反馈信息贡献约 57%，多采样贡献约 43%。部分设计题目在 15 次独立重试中均未通过，但在反馈条件下 2-3 轮即可修复。

稳定性实验（4 轮重复）支持了核心结论的可复现性。反馈粒度实验表明编译反馈和简洁反馈最为稳健，过于详尽的反馈反而可能干扰模型的修复判断。实验还发现反馈效果并非对所有模型普适有效，Sonnet 4.6 上反馈未能超越纯重试，不同模型对反馈信息的利用效率存在差异。

关键词：大语言模型；Verilog 代码生成；EDA 工具反馈；自动优化；RTLLM



LLM Verilog Generation and Automatic Optimization Based on EDA Tool Feedback

Author: Zhang Jintao

Tutor: Wang Rui

Abstract

Large Language Models can now generate Verilog code from natural language descriptions, yet their zero-shot outputs frequently contain compilation errors or simulation mismatches on complex designs involving finite state machines, precise bit-width operations, and algorithmic RTL designs. This thesis asks whether EDA tool outputs—compilation and simulation results already available in the design flow—can guide models toward targeted repair of such errors.

A prototype system was built that parses Icarus Verilog compiler and VVP simulator outputs into structured feedback signals, driving the LLM through an iterative generate–compile–simulate–feedback–repair loop. The system supports two public benchmarks (VerilogEval-Human and RTLLM), multi-model switching via a unified API gateway, five levels of feedback granularity, and four prompt engineering strategies.

Seven progressive experiments were conducted across three models spanning different capability levels. On VerilogEval-Human (20-problem subset), Haiku’s pass rate improved from 80% to 90%. On the more demanding RTLLM_STUDY_12 (12 curated designs), GPT-5.4 rose from 50% to 83%. Ablation experiments decomposed the improvement: error feedback contributed roughly 57%, while multi-sampling accounted for 43%. Some designs could not be solved by 15 independent retry attempts but were repaired within 2–3 feedback rounds.

Four independent repetitions supported the reproducibility of the core findings. A granularity sweep showed that compile-only and succinct feedback proved the more robust choices, while excessively detailed feedback risked degrading repair reasoning. Feedback effectiveness was model-dependent: on Sonnet 4.6, feedback did not outperform pure retry, showing that the benefit of feedback loops does not generalize to all models we tested.



Key words: Large Language Models; Verilog Generation; EDA Tool Feedback; Automatic Optimization; RTLLM



目 录

1 绪论	4
1.1 研究背景与意义	4
1.2 国内外研究现状	4
1.2.1 LLM 辅助 RTL 代码生成	4
1.2.2 EDA 工具反馈与自动修复	5
1.2.3 现有工作的不足	5
1.3 本文研究内容	5
1.4 本文主要贡献	6
1.5 论文组织结构	6
2 相关技术与研究基础	7
2.1 Verilog 与 RTL 设计基础	7
2.1.1 RTL 设计的基本概念	7
2.1.2 RTL 代码与软件代码的关键差异	7
2.2 大语言模型代码生成基础	8
2.2.1 从 Transformer 到代码生成与指令跟随	8
2.2.2 LLM 生成 RTL 代码的常见问题	8
2.3 EDA 编译与仿真验证流程	8
2.3.1 编译验证	8
2.3.2 功能仿真	9
2.3.3 编译仿真反馈的信号价值	9
2.4 AutoChip 方法与 EDA Feedback Loop	9
2.4.1 AutoChip 的核心思想	9
2.4.2 反馈信息的组织	10
2.4.3 多候选生成与全局最优追踪	10
2.4.4 本文工作与最接近相关工作的关系	10



2.5 RTL 生成评估基准	11
2.5.1 VerilogEval Benchmark	11
2.5.2 RTLLM Benchmark	11
2.6 相关研究与本文定位	11
2.6.1 相关研究方向	11
2.6.2 本文定位	12
2.7 本章小结	12
3 系统设计与实现	13
3.1 系统目标与关键挑战	13
3.2 总体方案：EDA 反馈驱动的迭代修复闭环	14
3.3 多源 benchmark 的统一接入	14
3.4 代码生成与提取	17
3.5 编译仿真与评分	19
3.6 反馈构造与迭代修复控制	20
3.7 实验配置扩展与结果管理	24
3.8 本章小结	24
4 实验设计	26
4.1 实验目标与总体设计	26
4.2 评估基准与任务子集	26
4.3 模型与参数设置	28
4.4 实验条件与递进逻辑	29
4.4.1 第一层：有效性验证	29
4.4.2 第二层：收益归因与稳定性	29
4.4.3 第三层：机制探索	29
4.4.4 实验设计与第 5 章结果章节的对应关系	30
4.5 评价指标与统计方法	31
4.6 实验可靠性控制	31
4.7 本章小结	31



5 实验结果与分析	32
5.1 VerilogEval-Human 基线实验	32
5.2 RTLLM_STUDY_12 正式实验结果	32
5.3 Feedback vs Retry-only 消融实验	34
5.4 稳定性重复实验	36
5.5 反馈粒度实验	37
5.6 多轮对话反馈实验	39
5.7 提示策略实验	40
5.8 典型案例分析	41
5.9 综合讨论	42
5.10 本章小结	45
6 总结与展望	46
6.1 本文工作总结	46
6.2 主要实验结论	46
6.3 系统局限性	46
6.4 后续工作展望	47
6.5 本章小结	47
结论	48
致谢	49
参考文献	50
附录 A 实验配置与运行命令	53
A.1 正式实验运行命令	53
A.2 环境配置	54
A.3 核心代码模块索引	54
附录 B 补充实验图表	55



1 绪论

1.1 研究背景与意义

随着集成电路设计规模的持续增长，寄存器传输级（RTL）代码的编写和验证已成为芯片设计流程中最耗时的环节之一。传统的 RTL 开发高度依赖工程师的专业经验——设计者需要根据功能规格手工编写 Verilog 或 VHDL 代码，并通过反复的编译、仿真和调试迭代来确保功能正确性。这一过程周期长、人力成本高，且容易因疏忽引入难以发现的逻辑错误。

近年来，大语言模型（Large Language Model, LLM）在软件代码生成领域展现出一定能力，能够根据自然语言描述生成多种编程语言的代码。这一进展自然引出了一个研究问题：大语言模型能否有效地生成硬件描述语言代码，从而辅助甚至部分替代人工的 RTL 开发过程？

然而，RTL 代码生成与普通软件代码生成存在本质差异。RTL 代码描述的是并行执行的硬件结构而非顺序执行的软件逻辑，正确的 RTL 设计不仅要求语法正确，还必须满足时序约束、位宽匹配、时钟域一致性等硬件特有要求，并通过 EDA 工具的编译和仿真验证。在零样本条件下，大语言模型生成的 RTL 代码往往存在编译错误、接口不匹配、边界条件遗漏等问题，尤其在涉及复杂状态机或浮点运算等场景时正确率较低。

这一现实使得将大语言模型的生成能力与 EDA 工具的验证能力相结合成为一个值得探索的方向。通过构建“生成—编译—仿真—反馈—修复”的反馈循环，EDA 工具的编译和仿真输出可以作为结构化的反馈信号引导模型定向修复代码错误，有望提升 RTL 代码的自动生成质量。

1.2 国内外研究现状

1.2.1 LLM 辅助 RTL 代码生成

已有研究探索了使用大语言模型从自然语言规格直接生成 Verilog 代码的可行性 [1-2]。在评估基准方面，NVlabs 发布的 VerilogEval[3] 提供了标准化的 spec-to-RTL 评估框架，香港科技大学发布的 RTLLM[4] 则提供了更高难度的设计任务集。这些基准的建立推动了 RTL 生成方法的标准化评估和横向比较。



现有研究表明,大语言模型在简单的组合逻辑和常见时序电路上具备一定的生成能力,但在涉及复杂算法、多状态 FSM 或精确位操作的设计上,单次生成的正确率仍有较大提升空间。

1.2.2 EDA 工具反馈与自动修复

AutoChip[5]等工作提出了利用 EDA 工具反馈驱动代码修复的方法,将传统的单次生成扩展为迭代修复过程。该类方法的核心思想是将编译器和仿真器输出的错误信息提取为结构化反馈,使模型能够进行定向修复而非盲目重新生成。RTLFixer[6]针对编译错误修复进行了专门探索,HDLLDebugger[7]引入了检索增强技术辅助调试,这些研究共同体现了“验证反馈驱动修复”的研究趋势。

1.2.3 现有工作的不足

尽管上述研究在方法探索和概念验证方面取得了有价值的进展,仍有几个方面值得进一步分析。

在反馈收益的来源方面,反馈循环带来的通过率提升中有多少来自错误信息的引导、有多少仅来自多次重试的采样收益,现有工作较少对此进行严格的消融分析。在强模型与高难度场景方面,部分研究基于较早的模型或较简单的 benchmark 进行评估,需要新的实验证据。在反馈机制的影响因素方面,反馈粒度、反馈组织方式和初始提示策略等维度对修复效果的影响,尚未得到系统的实验研究。此外,部分工作在实验结果管理和结论稳定性验证方面的机制不够完善。

1.3 本文研究内容

针对上述不足,本文的工作包含系统构建和实验研究两部分。

在系统构建方面,本文实现了一个基于 EDA 工具反馈的 LLM Verilog 生成与自动优化原型系统。该系统以 Python 编写,以 Icarus Verilog 作为 EDA 后端,实现了完整的“生成—编译—仿真—反馈—修复”反馈循环,并支持 VerilogEval 和 RTLLM 双 benchmark 适配、统一 API 网关下的多模型调用、五级反馈粒度和四种提示策略的灵活组合。此外,对 RTLLM 2.0 全部 50 个设计进行了 Icarus Verilog 兼容性审计(41/50 完全可用),从中精选 12 道高难度设计构成 RTLLM_STUDY_12 正式研究子集。

在实验研究方面,使用 Haiku、Sonnet 4.6 和 GPT-5.4 三个不同能力等级的模型,执



行了七组递进式对照实验。实验从基础有效性验证起步,经由 `retry-only` 消融实验分离多采样与反馈信息各自的贡献,进而通过 4 轮重复实验检验结论的统计稳定性。在此基础上,分别探索了反馈粒度、多轮对话模式和提示策略对修复效果的影响。全部实验均配有自动化产物管理和审计机制。

1.4 本文主要贡献

本文的主要贡献包括以下几个方面。首先,实现了一个基于 EDA 工具反馈的 RTL 生成与优化原型系统,支持 VerilogEval 与 RTLLM 双 benchmark 适配、多模型调用、五级反馈粒度与四种提示策略,并内置了实验产物自动序列化和数据审计脚本,使每次实验可追溯至具体代码版本。围绕核心系统,还实现了 RTLLM 兼容性扫描、多模式实验运行和结果图表整理等配套脚本。

其次,以 RTLLM_STUDY_12 为核心任务集,通过 `retry-only` 消融实验将反馈收益定量分解为多采样和错误反馈信息两部分,并通过 4 轮独立重复确认了结论的可复现性。实验中发现反馈效果具有模型依赖性:GPT-5.4 上反馈始终带来正向增益,而 Sonnet 4.6 上反馈在当前配置下未能超越纯重试。

此外,五级粒度实验表明反馈信息量与修复效率之间并非单调递增关系——编译反馈和简洁反馈的稳健性最强,过于详尽的反馈可能引起信息过载。

1.5 论文组织结构

本文共分六章。第 1 章为绪论,介绍研究背景、现状和本文工作。第 2 章为相关技术与研究基础,介绍 Verilog 与 RTL 设计基础、大语言模型代码生成机制、EDA 编译仿真流程和反馈循环方法。第 3 章为系统设计与实现,描述原型系统的架构和关键模块。第 4 章为实验设计,定义全部实验方案和评价指标。第 5 章为实验结果与分析,呈现七组实验的结果并进行综合讨论。第 6 章为总结与展望,总结工作、讨论局限性并展望后续方向。



2 相关技术与研究基础

本章介绍与本文研究密切相关的技术基础和评估基准。内容涵盖 Verilog 硬件描述语言的设计要素、大语言模型的代码生成机制、EDA 工具链的编译仿真流程、反馈循环式 RTL 代码修复的相关工作（其中 AutoChip 是与本文最为接近的代表性工作），以及本文实验所依赖的两个公开评估基准。上述内容为后续第 3 章的系统设计和第 4-5 章的实验分析提供技术背景。

2.1 Verilog 与 RTL 设计基础

2.1.1 RTL 设计的基本概念

寄存器传输级（Register Transfer Level, RTL）是数字集成电路设计中广泛采用的抽象层次 [8]。在 RTL 层面，设计者以硬件描述语言刻画数据在寄存器之间的传输路径和变换逻辑，综合工具随后将其转化为门级网表，作为后续物理设计流程的输入。IEEE 1364-2005 标准 [8] 定义了 Verilog 语言的语法和语义规范，为 RTL 设计提供了工业级的标准化基础。

Verilog 的基本设计单元是模块（module）。每个模块声明一组输入/输出端口，并在模块体内实现硬件功能。按照信号驱动方式的不同，模块内部逻辑可分为两类：组合逻辑（其输出仅由当前输入决定，如加法器和多路选择器）与时序逻辑（其输出还受到过去状态的影响，如计数器和有限状态机）。有限状态机（FSM）是时序逻辑的典型代表，在协议解析、序列检测等数字系统控制场景中有广泛应用。

2.1.2 RTL 代码与软件代码的关键差异

RTL 代码的自动生成比普通软件代码更具挑战性，根本原因在于两者的执行模型截然不同。C/Python 等软件语言的语句按程序顺序依次执行，而 Verilog 中的 always 块和连续赋值 assign 在仿真语义下并发执行，正确生成 RTL 代码必须同时处理三类硬件特有的语义：信号之间的并发时序依赖、由时钟边沿触发并依赖复位完成初始化的时序逻辑行为、以及具有确定位宽的硬件信号上的符号扩展、截断与位拼接。这些语义在 LLM 生成 Verilog 时也是高频错误来源——遗漏 posedge clk 敏感列表项、缺少复位分支或位宽不匹配都会直接导致仿真结果与预期不一致。这些差异决定了 RTL 代码生成不能简



单套用软件代码生成的范式，EDA 工具的编译和仿真验证环节不可或缺。

2.2 大语言模型代码生成基础

2.2.1 从 Transformer 到代码生成与指令跟随

当代大语言模型基于 Transformer 自注意力机制 [9] 构建，并通过两个阶段获得本文实验所需的能力：以 GPT-3[10] 为代表的大规模无监督预训练让模型具备了从自然语言描述生成代码的基础能力，以 InstructGPT[11] 为代表的基于人类反馈的强化学习 (RLHF) 则使模型学会理解并遵循用户指令。在代码领域，OpenAI 的 Codex[12] 通过在 GitHub 公开代码库上对 GPT 微调显著提升了代码生成质量，并提出了 HumanEval 基准和 pass@k 评估指标——pass@k 衡量在 k 次独立采样中至少出现一个正确解的概率，本文的多候选生成策略 ($k = 3$) 即沿用这一思路。Meta 的 Code Llama[13] 进一步表明开源代码模型在 HumanEval 上也能达到接近闭源模型的水平。在本文的实验场景中，模型既需要从自然语言硬件功能描述生成符合规格的 Verilog 代码，也需要在迭代修复中理解反馈中的修复指令，预训练打下的代码生成能力和指令跟随对齐都直接影响首轮生成与后续修复的质量。

2.2.2 LLM 生成 RTL 代码的常见问题

尽管大语言模型具备一定的 Verilog 代码生成能力，实际使用中仍频繁出现各种错误。最常见的是直接导致 EDA 编译失败的语法和结构性问题——模块声明不完整、端口列表与设计描述不匹配、信号未声明等；其次是模块名称或端口与测试平台不一致导致的实例化失败；功能性问题则表现为代码在多数测试向量上正确但在特定边界条件上输出错误，或对复杂数学运算与状态转移逻辑的实现不够精确。这些问题在难度较高的设计任务上尤为突出，也为引入 EDA 编译和仿真反馈来迭代修复代码提供了直接动机。

2.3 EDA 编译与仿真验证流程

2.3.1 编译验证

硬件设计的编译阶段将 Verilog 源代码解析为仿真模型，并检查语法正确性、信号声明一致性和模块接口匹配性。编译器输出的错误信息能够精确定位代码中的结构性问题，例如未声明信号、端口数量不匹配或语法关键字拼写错误等。



本文选用 Icarus Verilog[14] 作为编译和仿真工具。Icarus Verilog 是一个遵循 IEEE 1364-2005[8] 标准的开源 Verilog 编译器与仿真器, 通过 -g2012 选项还可启用部分 SystemVerilog 扩展特性。选择开源 EDA 工具的考虑包括: 免费获取, 适合本科毕设的硬件和许可条件; 命令行接口简洁, 易于嵌入自动化脚本; 实验结果完全可复现。在工业实践中, Yosys[15] 等开源综合工具也已在学术界和小型 FPGA 项目中得到采用, 形成了从编译仿真到逻辑综合的开源 EDA 工具链。

2.3.2 功能仿真

编译成功后, 使用 VVP 仿真引擎执行编译产生的仿真二进制文件。仿真过程中, 测试平台 (testbench) 对待测设计施加激励信号并将输出与预期结果逐一比对。

VerilogEval 和 RTLLM 的测试平台采用了不同的输出格式: VerilogEval 通过逐样本比较待测模块与参考模块的输出并报告 “mismatched samples is N out of M”; RTLLM 则直接输出 “Your Design Passed” 或失败统计 “Test completed with N/M failures”。这些结构化的仿真输出使反馈提取成为可能。

2.3.3 编译仿真反馈的信号价值

编译器和仿真器的输出天然携带有价值的调试线索: 编译错误精确标识出语法或结构层面的缺陷位置和类型, 仿真不匹配则揭示逻辑层面的功能偏差。将这些信息结构化后反馈给大语言模型, 可以使模型的修复行为从盲目重写转变为定向修补。这一反馈循环思想构成了 AutoChip 方法的基础。

2.4 AutoChip 方法与 EDA Feedback Loop

2.4.1 AutoChip 的核心思想

AutoChip[5] 提出了一种基于 EDA 工具反馈的 Verilog 代码迭代修复方法。其核心流程为: 大语言模型根据自然语言描述生成初始 Verilog 代码 → EDA 工具编译并仿真验证 → 提取错误信息构造结构化反馈 → 将反馈与任务描述合并为新提示词 → 模型生成修复版代码。此过程反复执行, 直至代码通过全部测试或迭代次数耗尽。

与单次零样本生成相比, 该反馈循环具有两方面优势: 其一, 利用 EDA 工具提供的精确错误定位信息, 避免模型在已正确的部分做不必要的修改; 其二, 通过多轮迭代逐步逼近正确解, 在模型首次生成 “接近正确但存在局部错误” 的情况下尤为有效。



2.4.2 反馈信息的组织

反馈信息的组织粒度直接影响模型的修复效率。一个极端是将完整的编译日志和全部仿真波形传递给模型，但这会消耗大量上下文窗口并可能引入噪声。另一个极端是仅告知“编译失败”或“仿真未通过”，但这缺少足够的修复线索。AutoChip[5] 采用的简洁反馈策略介于两者之间：提取错误行号、不匹配样本数和仿真输出摘要，在控制输入长度的同时保留关键信息。RTLFixer[6] 的实践表明，仅针对编译错误的反馈就能修复相当比例的常见语法问题；HDLDebugger[7] 则通过检索增强技术，在反馈中引入了类似错误的历史修复案例，进一步提升了反馈的信息价值。

2.4.3 多候选生成与全局最优追踪

反馈循环不仅需要合理的反馈内容，还需要决定下一轮基于哪个候选继续修复。大语言模型的生成具有随机性，单次生成的代码质量波动较大。如果每轮只生成一个候选，修复过程容易因某一轮的低质量生成而误入歧途。因此，多候选生成与全局最优追踪机制的设计就显得尤为重要，这也与代码生成领域 $\text{pass}@k$ 评估方法 [12] 的思想相通。

在每轮迭代中生成多个候选代码 (k -candidate)，并从中选出编译/仿真评分最高的候选作为当前最优。全局最优追踪机制跨轮次维护历史最佳候选，确保下一轮反馈基于历史最优的错误信息构建，防止因单轮质量波动而丢失已有的最优解。

2.4.4 本文工作与最接近相关工作的关系

在已有研究中，AutoChip 与本文最为接近：二者都关注如何利用 EDA 工具的编译与仿真反馈引导大语言模型修复 Verilog 代码。与 AutoChip 侧重于展示反馈循环的可行性不同，本文面向 Verilog 代码生成与自动优化任务，构建了一个可复现的实验系统，并在此基础上从更难基准、多模型对比、反馈收益归因和反馈策略边界等维度展开进一步实验：在评估范围上，同时接入 VerilogEval 和 RTLLM 两个公开 benchmark；在模型覆盖上，使用 Haiku、Sonnet 4.6 和 GPT-5.4 三个不同能力层级的模型进行交叉验证；在实验方法上，设计了 `retry-only` 消融条件以定量分离多采样收益与反馈信息收益。

此外，本文还通过 4 轮独立重复实验验证结论的统计稳健性，并在反馈粒度、多轮对话和提示策略等维度进行了系统探索。AutoChip 在本章及第 5 章中作为最接近的相关工作和对比对象出现，而非本文系统的实现框架。



2.5 RTL 生成评估基准

2.5.1 VerilogEval Benchmark

VerilogEval[3] 是 NVIDIA 研究团队发布的 Verilog 代码生成评估基准，发表于 IC-CAD 2023。其人工编写子集（VerilogEval-Human）从 HDLBits[16] 在线练习平台中选取了 156 道硬件设计任务，涵盖组合逻辑、时序逻辑、有限状态机等多种类型，难度从基础的线连接到 Conway 生命游戏等复杂设计。VerilogEval-Human v2[17] 进一步引入了规格到 RTL（specification-to-RTL）的评估范式。

VerilogEval 的评估方式为：给定自然语言描述和模块接口定义，要求模型生成完整的模块实现代码。本文选取 VerilogEval-Human 的 20 道题目作为基线实验任务集。

2.5.2 RTLLM Benchmark

RTLLM[4] 是香港科技大学发布的 RTL 设计生成评估基准，发表于 ASP-DAC 2024。该基准包含 50 个硬件设计任务，按功能类别划分为算术运算（Arithmetic）、控制逻辑（Control）、存储器（Memory）和杂项（Miscellaneous）四大类，难度从基础加法器到 IEEE 754 浮点乘法器、流水线乘法器和交通灯控制器等工程复杂度较高的设计。

相比 VerilogEval，RTLLM 的任务普遍在模块规模和算法复杂度上更具挑战性。本文在接入 RTLLM 前，对全部 50 个设计进行了 Icarus Verilog 兼容性审计：50 个设计中 46 个编译通过（92%），41 个仿真通过（82%）。在 41 个完全可用的设计中，精选了 12 道高难度、多类别覆盖的设计构成 RTLLM_STUDY_12 正式研究子集。

2.6 相关研究与本文定位

2.6.1 相关研究方向

围绕大语言模型与硬件设计的交叉领域，当前研究主要分布在以下几条技术路线上。

在 LLM 直接生成 RTL 方面，ChipGPT[1] 探索了从自然语言到 RTL 的端到端生成流程，Verigen[2] 在 DATE 2023 上展示了通过微调生成 Verilog 的可行性，RTLCoder[18] 则通过专门的 RTL 代码微调提升了开源模型的 RTL 生成质量。在通用代码生成领域，Austin 等人 [19] 的研究表明程序合成性能随模型规模呈对数线性增长，AlphaCode[20] 则在竞赛级别代码生成上展示了大规模采样加程序行为过滤的有效性。在评估基准方面，



VerilogEval[3] 和 RTLLM[4] 的相继发布为该领域建立了标准化的评测框架。在 EDA 反馈驱动的代码修复方面, AutoChip[5] 提出了反馈循环范式, RTLFixer[6] 针对编译错误修复做了专门研究, HDLDebugger[7] 引入了检索增强技术辅助硬件代码调试, AIvrl[21] 则探索了更精细的 verification-in-the-loop 策略。在更广泛的代码修复领域, Self-Refine[22] 将迭代自反馈机制应用于多个任务并平均提升了 20%, Reflexion[23] 则通过语言式反馈记忆在 HumanEval 上达到了 91% pass@1, 这些工作共同表明了反馈驱动修复的广泛适用性。在多轮交互与代理方法方面, VerilogCoder[24] 采用图规划和 AST 工具链实现了更复杂的代码修正流程, Chip-Chat[25] 讨论了对话式硬件设计的挑战与机遇。AutoBench[26] 则将 LLM 应用于测试平台的自动生成。

2.6.2 本文定位

本文的定位是: 面向 Verilog 代码生成与自动优化任务, 构建一个由 EDA 编译与仿真反馈驱动的实验系统, 并将 AutoChip 等反馈式代码修复工作作为最接近的相关工作进行对比。在更强模型和更高难度的 benchmark 上, 本文系统地分析反馈循环的有效性及其影响因素。与上述相关工作相比, 本文的特色在于: 通过严格的消融实验设计分离反馈信息与多采样各自的贡献, 通过多轮独立重复实验验证结论的统计稳定性, 并在反馈粒度、多轮对话和提示策略等维度对反馈机制的适用条件和边界进行系统验证。

2.7 本章小结

本章介绍了本文工作所涉及的技术背景。从 RTL 设计和 EDA 工具链出发, 说明了 Verilog 代码正确性验证的基本流程; 随后介绍了大语言模型在代码生成方面的技术基础, 包括预训练和指令对齐机制; 在此基础上, 梳理了 AutoChip 等工作提出的反馈循环思想, 以及 VerilogEval 和 RTLLM 两个评估基准。这些技术背景构成了下一章系统设计的出发点: 如何将 LLM 的生成能力和 EDA 工具的验证能力结合成一个可用的自动化系统。



3 系统设计与实现

3.1 系统目标与关键挑战

本文的工程目标是构建一个能够自动完成“Verilog 代码生成—EDA 工具验证—错误反馈—代码修复”反馈循环的原型系统。给定一道硬件设计任务的自然语言描述和模块接口，系统应当能够调用大语言模型生成候选 Verilog 代码，借助开源 EDA 工具自动编译和仿真验证，从验证结果中提取错误信息构造反馈，并驱动模型进行迭代修复，直至代码通过全部测试用例或迭代次数耗尽。

在实现这一目标的过程中，需要解决三类工程挑战。

首先是外部异构输入的清洗与统一。一方面，VerilogEval[3] 和 RTLLM[4] 在文件组织、题目描述方式和测试平台输出格式上存在差异，如果不加处理，后续的生成、验证和反馈逻辑都需要写两套；另一方面，大语言模型的回复往往包裹在 Markdown 代码围栏中，使用 CoT 提示策略时还会穿插解释性文本，直接保存模型的完整回复无法通过 EDA 工具编译。

其次是 EDA 验证结果到反馈提示词的转化与平衡。编译失败、仿真失败、部分通过和完全通过是性质不同的信号，且 VerilogEval 与 RTLLM 的仿真输出格式也不同，系统需要一个统一的评价标准来判断候选代码的质量并选择当前最优候选进入下一轮反馈；在评分之上，反馈信息还需在信息量和噪声之间取得平衡——将完整的编译日志和仿真波形原样传递给模型会消耗大量上下文窗口并引入噪声，仅告知“编译失败”又缺少足够的修复线索，不同模型对反馈粒度的敏感程度也不同，还需支持多种反馈模式的灵活切换。

最后是实验规模增大后的可复现性保证。多模型、多条件、多轮实验会产生大量中间产物，如果缺少统一的记录和追溯机制，容易在论文撰写中误用已废弃的结果，项目中多轮对话实验从 v1 到 v2 的修正经历即说明了结果追溯的必要性。

下文围绕这三类挑战，依次介绍系统的总体方案和各部分的设计与实现。核心系统代码约 2100 行（含主代码与实验运行脚本，不含测试和配置文件），以 Python 实现，使用 Icarus Verilog[14] 作为 EDA 后端。此外还配套实现了 RTLLM 兼容性扫描、多模式实验调度和结果图表整理等脚本，为后续实验提供工程支撑。



3.2 总体方案：EDA 反馈驱动的迭代修复闭环

系统的总体方案是将代码生成任务组织为一个反馈循环：每轮迭代中，模型根据提示词生成候选代码，EDA 工具对候选代码进行编译和仿真，评分器将验证结果映射为标量分数，反馈构造器从验证结果中提取关键错误信息，控制器判断是否需要继续迭代并组装下一轮的提示词。这一反馈循环的基本前提是：EDA 工具能够提供精确的错误定位信息，而大语言模型能够利用这些信息进行定向修复。

为实现上述反馈循环，系统划分为八个核心模块，各模块的职责和对应代码文件如表3.1所示。数据在模块间的流转路径为：任务加载器 → 提示词构建器 → LLM 客户端 → Verilog 提取器 → Verilog 执行器 → 评分器 → 反馈循环控制器 → 产物管理器。图3.1展示了模块间的调用关系。

表 3.1 系统核心模块与代码文件映射

模块	职责	代码文件
任务加载器	从 benchmark 目录读取题目	rtllm_loader.py, verilogeval_loader.py
提示词构建器	组装初始/反馈提示词	prompt_builder.py
LLM 客户端	封装 API 调用与重试	client.py
Verilog 提取器	从模型回复中提取代码	extract_verilog.py
Verilog 执行器	调用 iverilog/vvp	verilog_executor.py
评分器	编译/仿真结果 → 标量分数	ranker.py
反馈循环控制器	反馈循环主逻辑	loop_runner.py
产物管理器	元数据记录与序列化	artifacts.py

在工程约束方面，系统需要运行在本科毕设可获取的软硬件环境下：不依赖商业 EDA 工具许可，不要求 GPU 本地部署，全部依赖均可通过 pip 安装。

3.3 多源 benchmark 的统一接入

针对 benchmark 格式不统一的问题，系统定义了统一的 Task 数据结构作为所有下游模块的输入接口。Task 包含四个必需字段：name（任务名称，如 adder_16bit）、description（自然语言功能描述）、module_header（Verilog 模块的端口声明）和 testbench_path（测试平台文件的绝对路径）。所有下游模块——提示词构建器、执行器、评分器等——仅依赖 Task 接口，不感知数据来源是 VerilogEval 还是 RTLLM。

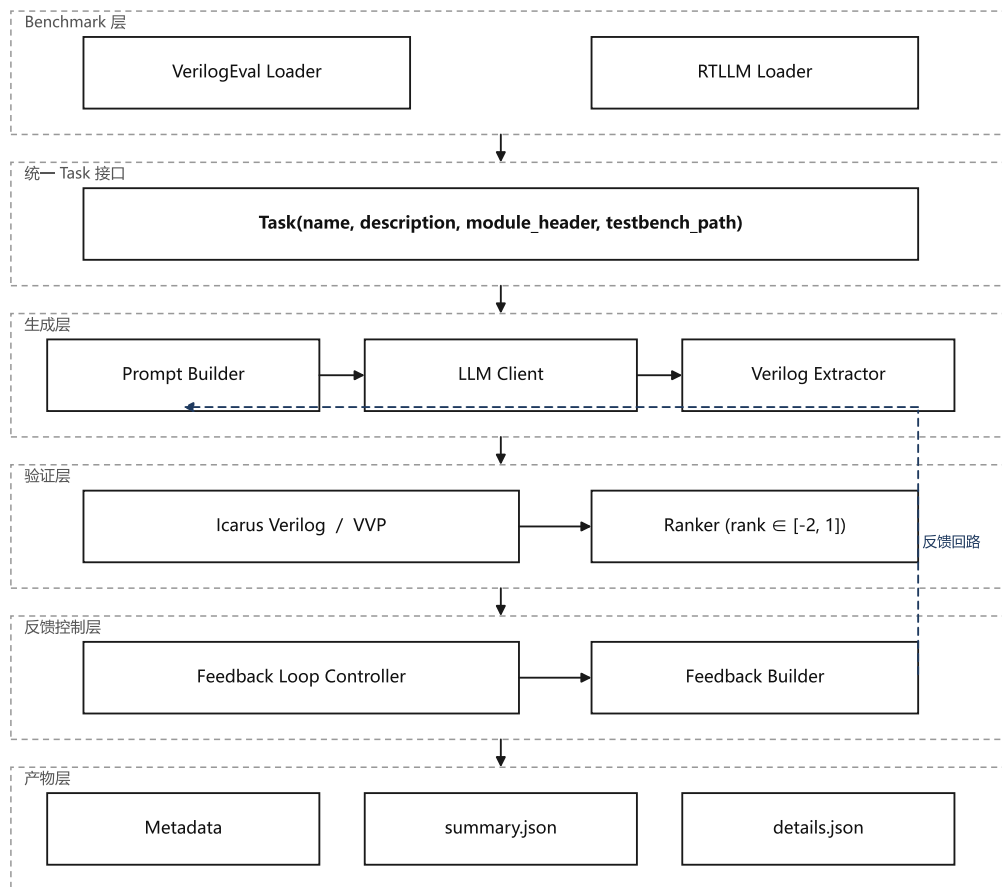


图 3.1 系统模块架构与数据流。系统按 Benchmark 加载、统一 Task 接口、生成、验证、反馈控制和产物输出六层组织：实线箭头表示前向数据流，深蓝色虚线表示反馈回路，由反馈构造器将验证结果回送至提示词构建器以进入下一轮迭代。

这一设计将 benchmark 差异封装在加载器内部。RTLLM 2.0 的题目分布在 Arithmetic、Control、Memory、Miscellaneous 四个分类目录下的嵌套子目录中，加载器通过递归遍历目录结构，对每个同时包含 design_description.txt 和 testbench.v 的子目录创建一个 Task 对象。模块名称从设计描述文件中通过正则表达式提取，提取失败时回退为默认值。VerilogEval 采用 spec-to-RTL 模式，每道题目提供自然语言描述和 TopModule 接口定义，加载器读取描述文本和接口声明，将参考实现与测试平台合并写入临时文件。两个加载器在输出端均产生统一的 Task 对象，使后续流程无需区分数据来源。

图3.2展示了从两类原始数据格式归一化到统一 Task 接口、再分发给下游模块的整体流程。两个 benchmark 在表层差异显著：VerilogEval-Human 的描述文本与 TopModule 接口分开存储、参考实现需与测试平台合并，而 RTLLM 2.0 则将描述放在 design_description.txt、测试平台直接以 testbench.v 提供。如果不做归一化，提示词构建器、执行器、评分器都需要分别处理两套数据来源，每新增一个 benchmark 就要改动多处代码。统一 Task 接口将所有差异收敛到加载器一层，下游模块仅依赖 name/description/module_header/testbench_path 四个字段，为后续多 benchmark 交叉实验提供了工程基础。

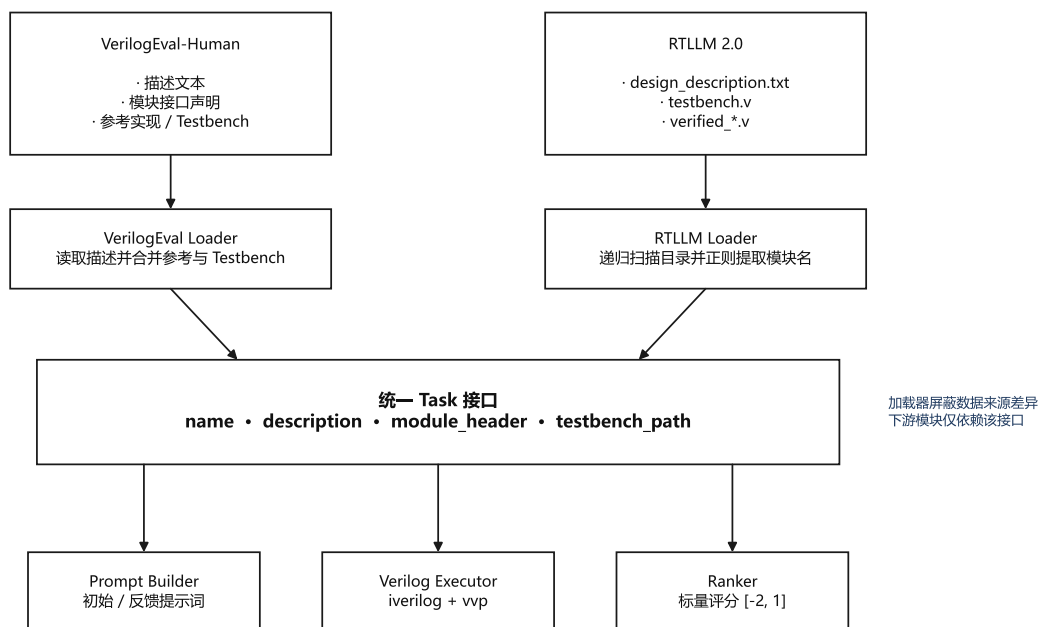


图 3.2 多源 benchmark 到统一 Task 接口的转换流程

这种统一接口的好处在实验中得到了直接验证：当项目从仅支持 VerilogEval 扩展



到同时支持 RTLLM 时, 生成、验证和反馈的代码无需任何修改, 只需新增一个加载器即可。后续在第 4 章对三个模型、七组实验、两个 benchmark 进行交叉对比时, 实验脚本也只需在命令行参数中切换 benchmark 名称, 模块层无需感知任何差异。

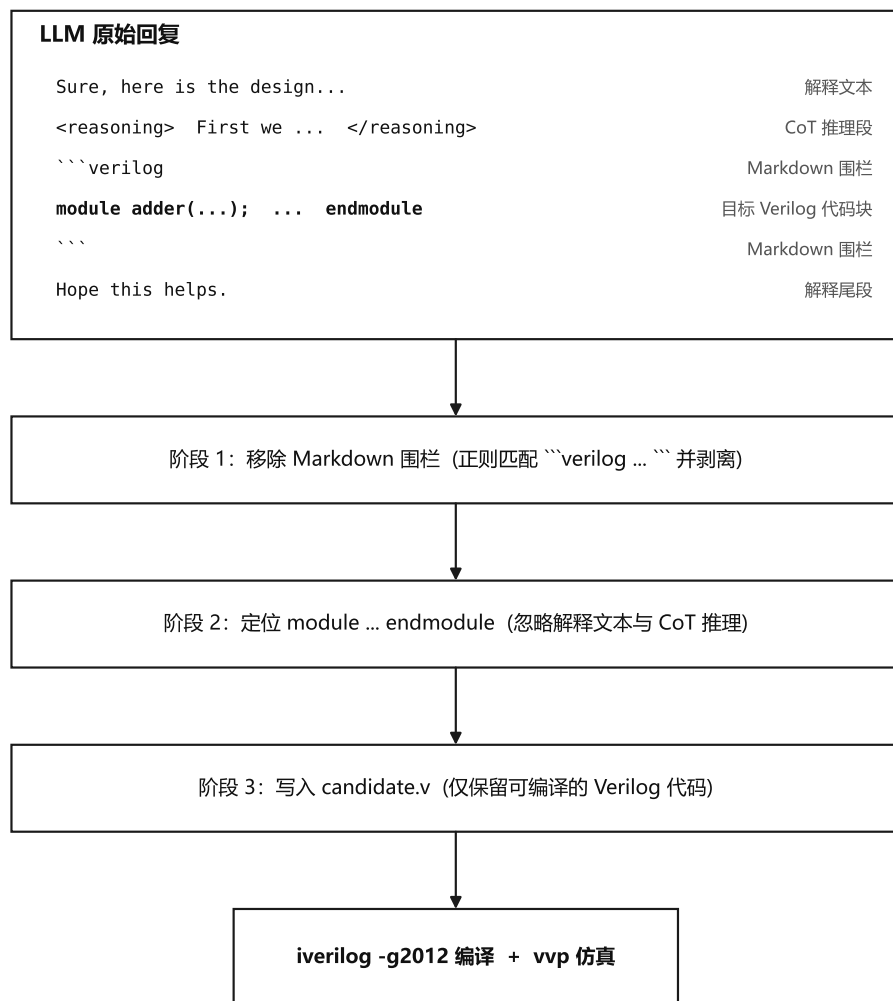
3.4 代码生成与提取

大语言模型的回复格式不可控, 这是将 LLM 接入自动化流水线时必须解决的问题。模型可能返回 Markdown 代码围栏包裹的代码、穿插在解释文本中的代码片段, 甚至在使用 CoT 策略时先输出一段推理过程再给出代码。如果不做提取处理, 这些回复无法直接交给 EDA 工具编译。

系统通过 `extract_verilog.py` 实现了两阶段代码提取。第一阶段使用正则表达式移除所有 Markdown 围栏标记 (“`verilog ...` “), 第二阶段通过 `module\s+\w+...endmodule` 模式匹配提取全部 Verilog 模块块。该策略对不同提示策略产生的回复格式均具有兼容性——无论模型是直接返回代码还是在代码前后添加解释文本, 提取器都能正确定位 module 到 endmodule 之间的有效代码。

图3.3给出了从原始 LLM 回复到可编译 Verilog 文件的完整处理流程。原始回复中常见的非代码内容包括: 开场客套语 (“Sure, here is the design...”), CoT 提示策略下的推理段 (包裹在 `<reasoning>` 标签或 “First we need to...” 之类自由形式的段落中)、Markdown 围栏与语言标签、以及结尾的解释文本。如果将整段回复直接送入 `iverilog`, 编译器会因遇到非 Verilog 语法而失败。两阶段提取的设计动机正是分离 “回复结构” 与 “代码内容”: 第一阶段只关心去除围栏, 第二阶段只关心从清理后的文本中匹配模块边界, 两个阶段彼此独立, 便于后续扩展 (例如未来若引入新的提示模板, 只需调整第一阶段的正则即可)。这一设计也使得提取器无需为 Base、CoT、Few-shot、Few-shot+CoT 四种提示策略 [27] 维护各自的解析器, 第5.7节的提示策略实验在统一提取流程下完成, 避免了因解析逻辑差异引入的混淆变量。

在模型调用层面, 系统通过第三方统一 API 网关接入多种大语言模型。网关使得 API 密钥和服务端点保持不变, 仅通过环境变量中的模型名称参数切换底层模型。这一设计使得在同一实验脚本中切换 Haiku、Sonnet 和 GPT-5.4 只需修改一个环境变量。针对 API 调用中不可避免的瞬态错误 (网络超时、服务端过载等), 系统实现了指数退避重试机制: 对可重试错误依次等待 2 秒、5 秒和 10 秒, 最多重试 3 次。当所有重试均失败时, 错误被记录但不会导致整个实验批次崩溃。



该提取流程兼容 Base / CoT / Few-shot / Few-shot+CoT 四种提示策略，无需为不同策略维护专用解析器。

图 3.3 LLM 回复到可编译 Verilog 文件的提取流程



3.5 编译仿真与评分

EDA 工具的输出需要转化为可比较的分数，这是实现自动化评估和反馈选择的前提。编译阶段调用 Icarus Verilog，命令为 `iverilog -g2012 -o sim.out design.v testbench.v`，其中 `-g2012` 选项启用 SystemVerilog 2012 扩展以支持 RTLLM 测试平台中使用的 logic 等新语法。编译超时设为 30 秒——RTLLM_STUDY_12 中实测最长编译时间约 11 秒，30 秒上限留有约 2.7 倍安全边界。

编译成功后，使用 VVP 仿真引擎执行编译产物，超时设为 60 秒。仿真输出的解析需要支持两种格式：VerilogEval 格式使用正则表达式匹配 `mismatched samples is N out of M` 模式提取不匹配数和总样本数；RTLLM 格式检测 `Your Design Passed` 或 `Test completed with N/M failures`。当两种格式均未匹配时，以进程返回码作为最终判断依据。

评分器将上述编译和仿真结果映射为 $[-2, 1]$ 区间内的标量分数（表3.2）。该评分规则参考 AutoChip[5] 的“分数越高代码质量越好”的总体取向，并在三个具体方面做了调整以适配本系统：用 -2.0 分独立标识“代码提取失败”以与“编译失败”区分，便于后续诊断 LLM 回复格式问题；将“编译有警告但无仿真结果”作为 -0.5 分的中间状态，避免单纯按“是否通过”的二值评分丢失部分通过的信息；将仿真部分通过映射为按比例连续分数 $(\text{总样本} - \text{不匹配}) / \text{总样本}$ ，使全局最优追踪能识别“接近正确”的候选。反馈循环控制器以 $\text{rank}=1.0$ 作为“通过”的判定阈值，统一的标量评分使得不同来源的候选代码可以直接比较，也为全局最优追踪提供了量化依据。

表 3.2 候选代码评分规则

情况	分数	含义
未提取到有效 Verilog	-2.0	代码提取失败
编译失败	-1.0	存在语法或结构错误
编译有警告但无仿真结果	-0.5	可能存在问题
编译成功但无仿真数据	0.0	无法评估正确性
仿真部分通过	$0 \sim 1.0$	$(\text{总样本} - \text{不匹配}) / \text{总样本}$
仿真完全通过	1.0	设计功能正确

需要说明两种中间状态的含义。“编译有警告但无仿真结果”（ -0.5 分）对应编译器报告了 warning 且仿真进程未能正常启动的情况，通常由使用了 Icarus Verilog 不完全支



持的语法特性引起。“编译成功但无仿真数据” (0.0 分) 则对应编译通过但仿真运行超时或测试平台未产生输出的情况, 在 RTLLM 部分题目的复杂测试平台中偶尔出现。

3.6 反馈构造与迭代修复控制

反馈信息的构造和迭代过程的控制是整个反馈循环的核心。反馈循环控制器(loop_runner.py) 负责组织“生成—验证—反馈—修复”的迭代流程, 其执行逻辑如算法1所示。

Algorithm 1 单轮反馈循环执行流程

Require: 任务 T (description, module_header, testbench_path)

Require: 参数 k (每轮候选数)、 N (最大迭代轮数)、 m (反馈模式)

Ensure: 最优 Verilog 代码及其评分

```

1:  $global\_best \leftarrow null, best\_rank \leftarrow -2.0$ 
2:  $prompt_0 \leftarrow BuildInitialPrompt(T)$ 
3: for  $i = 1$  to  $N$  do
4:   if  $i = 1$  or  $m = \text{retry-only}$  then
5:      $prompt \leftarrow prompt_0$ 
6:   else
7:      $prompt \leftarrow BuildFeedbackPrompt(T, global\_best, m)$ 
8:   end if
9:   for  $j = 1$  to  $k$  do
10:     $code_j \leftarrow ExtractVerilog(LLM(prompt))$ 
11:     $rank_j \leftarrow Evaluate(code_j, T.testbench)$ 
12:    if  $rank_j > best\_rank$  then
13:       $global\_best \leftarrow code_j, best\_rank \leftarrow rank_j$ 
14:    end if
15:  end for
16:  if  $best\_rank = 1.0$  then
17:    return ( $global\_best, 1.0$ ) {设计通过}
18:  end if
19: end for
20: return ( $global\_best, best\_rank$ ) {迭代耗尽}
```

控制器维护一个跨迭代的全局最优候选。每当某个候选的评分超过当前最优时, 控制器更新最优记录。下一轮迭代的反馈提示词始终基于全局最优候选的错误信息构建, 确保反馈循环具有单调性: 即使某一轮的候选质量暂时回退, 已经取得的最优结果也不会丢失。

零样本模式是反馈循环在 $k = 1$ 、 $\max_iterations=1$ 时的退化形式。Retry-only 模式 ($k = 3$ 、 $\max_iterations=5$ 但不注入反馈信息) 则在所有迭代中均使用初始提示词。这两

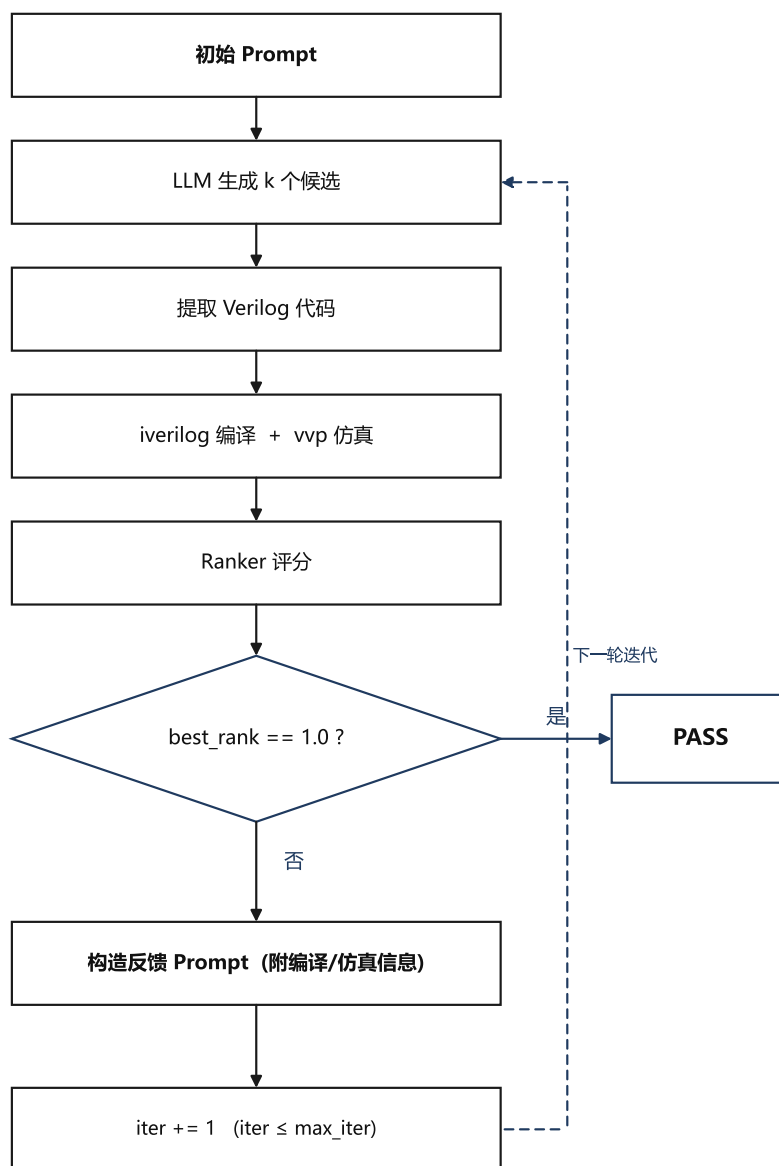


图 3.4 反馈循环控制流程（详细执行逻辑见算法1）



种模式与完整反馈模式共享同一代码路径，确保了实验条件的一致性和对比的公平性。当某个候选的 API 调用失败时，该候选被标记为错误但不终止当前迭代，只有一轮中全部候选都失败时循环才提前退出。

反馈信息的具体构造逻辑由评分结果驱动。图3.5展示了从 EDA 验证结果到下一轮反馈提示词的完整决策路径：编译器和仿真器的输出首先被评分器映射为 $[-2.0, 1.0]$ 区间内的标量分数（编译失败、编译警告、编译通过、仿真不匹配、仿真通过等情况各对应不同分值，详见表3.2）；控制器随后根据 rank 是否等于 1.0 判断当前是否已通过——若已通过则直接返回 global_best 并记录 PASS，否则将 rank 和原始 EDA 输出共同交给反馈构造器，由其按照配置的粒度模式生成简短或详细的反馈文本，再拼入下一轮 prompt。这一流程的关键在于：评分器对 rank 的统一量化使得不同来源的错误（编译错误、仿真不匹配、超时）能够进入同一比较空间，全局最优追踪也得以基于一个标量值进行；而反馈构造器与评分器的解耦则允许在不改动评分逻辑的前提下灵活切换 L2/L3/L4 三种反馈粒度，这一设计直接支撑了第5.5节的反馈粒度实验。

针对反馈信息量与噪声的平衡问题，系统实现了五个梯度的反馈水平（表3.3）。从不含任何反馈的 L0（零样本）到包含 80 行仿真输出和分析提示的 L4（丰富反馈），这五个级别覆盖了从”完全无信息”到”尽可能多信息”的完整频谱。五个级别共享相同的循环框架，仅在反馈提示词的构造方式上存在差异。L0 和 L1 通过循环控制器的参数控制（单次执行或禁用反馈注入），L2 至 L4 由提示词构建器中的 FeedbackMode 枚举实现。

表 3.3 反馈粒度级别定义

级别	名称	模式	反馈内容
L0	Zero-shot	无反馈	仅初始提示词，单次生成
L1	Retry-only	无反馈	每轮使用初始提示词，多次独立重试
L2	Compile-only	编译反馈	仅包含编译阶段错误信息
L3	Succinct	简洁反馈	编译错误 + 仿真不匹配数 + 输出前 40 行
L4	Rich	丰富反馈	编译错误 + 仿真输出前 80 行 + 分析提示

除单轮 API 调用模式外，系统还实现了多轮对话反馈。在该模式下，LLM 客户端维护一个持续增长的对话历史，每次 API 调用将完整历史传入，使模型能够看到之前的

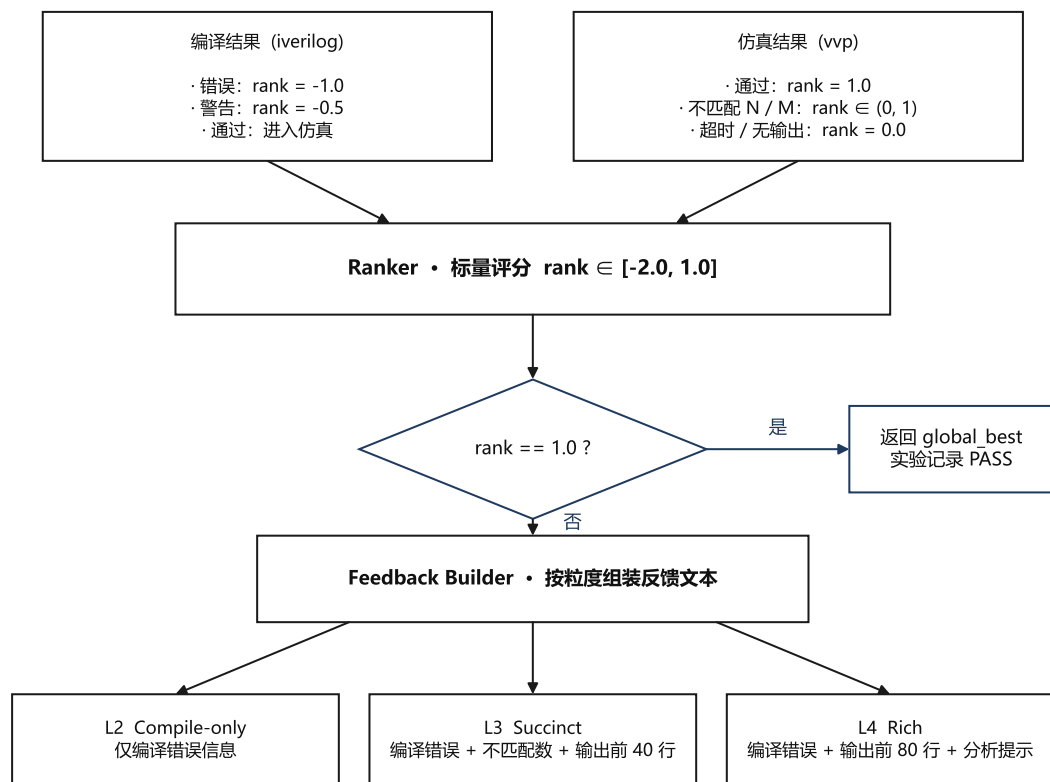


图 3.5 EDA 验证结果到反馈提示词的转换逻辑



生成和修复过程。多轮对话的关键约束是反馈消息必须描述模型在当前对话中最近一轮代码的错误，不能引用对话外的代码。该实现在初始版本中曾出现反馈信息与对话上下文不一致的 bug，导致实验数据失效；修复后的 v2 版本严格保证了一致性。

为研究初始提示词对 RTL 生成质量的影响，系统实现了四种可切换的提示策略 [27]：P0 (Base) 直接要求模型返回 Verilog 代码，P1 (CoT) 要求先推理再输出代码，P2 (Few-shot) 在提示词中附加两个示例，P3 (Few-shot+CoT) 结合示例和推理。示例选取避免了与正式实验任务重叠，两个示例均不在 RTLLM_STUDY_12 中。策略通过 PromptStrategy 枚举定义，策略名称自动记录到实验元数据中。

3.7 实验配置扩展与结果管理

实验规模增大后，结果的可复现性和可追溯性变得尤为重要。每次实验运行被分配一个唯一的 run_id (如 rtllm_feedback_20260420_153022)，产物存放在 outputs/runs/<类别>/<run_id>/下，包含三类文件：summary.json 记录运行级汇总，details.json 记录每道题每轮每个候选的完整信息，summary.csv 提供表格化的题目级结果。

产物管理器在每次运行开始时自动采集元数据，包括当前 Git commit hash、分支名称、工作区是否有未提交修改、Python 版本、操作系统平台和完整的命令行参数。这些信息使得任何实验结果都可以追溯到产生它的代码版本和运行环境。

项目设计了两个审计脚本来保障数据可信度。代码审计脚本 (audit_project.py) 围绕评分器、Verilog 模块提取器、仿真输出解析器和反馈循环逻辑执行 4 项核心检查，确保核心接口在迭代过程中不发生回归。数据审计脚本 (audit_data.py) 遍历全部实验运行目录，从原始 summary.json 中重新计算通过率并与文档记录的结论交叉验证。此外，项目维护一份实验结果索引文档，对每个实验运行标注其状态 (有效/已废弃/已替代)，避免论文撰写中误引用已废弃的结果。本文将各组实验的主要结果整理为通过率对比图、逐题矩阵和稳定性误差棒图等图表，供第 5 章结果分析引用。

3.8 本章小结

本章围绕“让 LLM 自动生成并修复 Verilog 代码”这一目标，针对外部异构输入的清洗、EDA 验证结果到反馈提示词的转化与平衡、以及实验可复现性三类工程挑战给出了系统设计与实现。其中两项核心设计决策值得重申：把 benchmark 差异封装在加载



器层、把 LLM 回复差异封装在提取器层，使下游的执行、评分、反馈构造和实验管理仅依赖 Task、Verilog 文本和 rank 三种与数据来源解耦的中间产物；将编译仿真结果统一量化为 $[-2, 1]$ 区间内的标量分数，使全局最优追踪和反馈选择得以基于一个标量值进行，反馈构造器与评分器的解耦也使 L0-L4 五级粒度切换和单/多轮对话切换能够在不改动主流程的前提下接入。这些设计是第 4 章实验能在三个模型、两个 benchmark、七组实验条件之间灵活组合的工程基础。



4 实验设计

本章定义全部实验的目标、评估基准、模型配置、实验条件和评价指标。实验设置均来自真实的实验脚本，具体结果将在第 5 章中呈现。

4.1 实验目标与总体设计

本文的实验围绕一个核心问题展开：EDA 反馈循环能否在不同难度的 benchmark 和不同能力的模型上稳定提升 Verilog 代码正确率？围绕这一核心问题，实验进一步探索反馈收益的来源分解、反馈粒度的影响和优化策略的效果。

为此，实验按照“先验证有效性、再分析收益来源、最后探索优化空间”的逻辑组织为三个层次。第一层为有效性验证：先在 VerilogEval-Human 上运行基线实验确认系统可以正常工作，再在难度更高的 RTLLM_STUDY_12 上用三个模型进行正式对比。第二层关注反馈收益的因果归因：通过 retry-only 消融条件将反馈收益分解为多采样和反馈信息两个来源，并用 4 轮独立重复检验结论是否稳定。第三层为机制探索：分别从反馈信息量、信息组织方式和初始生成质量三个角度分析可优化的空间。表 4.1 汇总了各研究问题与实验的对应关系。

表 4.1 研究问题与实验设计对应关系

编号	研究问题	对应实验	主要比较条件
RQ1	反馈循环是否提升正确率	基线 + 正式实验	Zero-shot vs Feedback
RQ2	反馈 vs 多次重试的贡献	消融实验	Retry-only vs Feedback
RQ3	结论是否统计稳定	稳定性重复	4 轮独立重复
RQ4	反馈粒度的影响	粒度实验	L0-L4 五级
RQ5	多轮对话是否优于单轮	多轮对话实验	ST vs MT
RQ6	提示策略的影响	策略实验	P0-P3 四种策略
RQ7	跨模型一致性	全部实验	三个模型交叉验证

4.2 评估基准与任务子集

实验使用两个公开 benchmark，分别承担不同的验证角色。

VerilogEval-Human[3] 是 NVIDIA 研究团队发布的 Verilog 代码生成评估基准的人



工编写子集，本文选取其中 20 个题目作为基线实验的任务集，覆盖组合逻辑、时序逻辑、有限状态机等多种类型。选择 VerilogEval-Human 作为基线的原因是其任务定义清晰、测试平台稳定，适合在系统开发初期验证反馈循环的基本功能是否正确。

导师建议引入更强模型和更难 benchmark 来验证结论，因此后续正式实验转向 RTLLM 2.0[4]。RTLLM 包含 50 个硬件设计任务，难度普遍高于 VerilogEval。由于本文使用 Icarus Verilog[14] 作为编译仿真工具，接入前对全部 50 个设计进行了兼容性审计（表4.2）。

表 4.2 RTLLM 2.0 Icarus Verilog 兼容性审计结果

指标	数量	比例
总设计数	50	100%
编译通过	46	92%
仿真通过	41	82%
编译失败	4	8%
编译通过但仿真异常	5	10%

不直接使用全部 50 题的原因有三：9 题因 Icarus Verilog 兼容性问题无法正确评估，余下 41 题全量运行的 API 成本较高且单次实验耗时长，不利于多模型多条件的重复实验。因此本文采取了分阶段选题策略：先用 5 题最小验证子集 RTLLM_CORE_5 确认系统在 RTLLM 格式下的端到端正确性（该子集结果仅作为工程验证，不进入正式结论），再从 41 题中精选 12 题构成正式研究子集 RTLLM_STUDY_12。

RTLLM_STUDY_12 的选取遵循三个标准：编译和仿真均通过兼容性审计；覆盖算术运算、控制逻辑、存储器和杂项功能四大类别；在算法复杂度、状态规模和数据通路类型上具备一定梯度，确保实验能够区分不同模型和条件的差异。其中 float_multi 涉及 IEEE 754 浮点规格化与尾数运算，是整个 RTLLM 中工程复杂度较高的设计之一；sequence_detector 和 freq_divbyfrac 则在预实验中被发现对所有模型均构成挑战。12 题的具体组成、所属类别和选入理由见表4.3。

表4.3不再对题目给出主观的星级排序，而是列出每道题的设计特征和选入理由，既能说明子集在数据通路、状态控制、存储管理和时序逻辑上的覆盖面，也避免用简单星级误导读者对任务相对难度的判断。



表 4.3 RTLLM_STUDY_12 任务组成与选入理由

#	设计名	类别	设计特征与选入理由
1	float_multi	Arithmetic/Other	浮点乘法, 覆盖规格化与尾数运算
2	multi_booth_8bit	Arithmetic/Multiplier	Booth 乘法器, 覆盖乘法数据通路
3	multi_pipe_8bit	Arithmetic/Multiplier	流水线乘法器, 覆盖时序数据通路
4	div_16bit	Arithmetic/Divider	16 位除法器, 覆盖迭代算术逻辑
5	adder_bcd	Arithmetic/Adder	BCD 加法, 覆盖位宽与进位处理
6	fsm	Control/FSM	有限状态机, 覆盖基本控制逻辑
7	sequence_detector	Control/FSM	序列检测, 覆盖状态转移与接口匹配
8	JC_counter	Control/Counter	计数器, 覆盖时序状态更新
9	LIFObuffer	Memory/LIFO	LIFO 缓冲区, 覆盖指针与存储控制
10	LFSR	Memory/Shifter	移位寄存器, 覆盖反馈多项式
11	traffic_light	Miscellaneous/Others	交通灯控制, 覆盖多状态时序控制
12	freq_divbyfrac	Miscellaneous/Freq divider	分数分频器, 覆盖精确时序与算法逻辑

4.3 模型与参数设置

实验选择三个大语言模型, 代表三个不同的能力等级 (表4.4)。Haiku 定位为较弱基线, 用于 VerilogEval 基线实验和 RTLLM 正式对比; Sonnet 和 GPT-5.4 定位为中强和强模型, 参与全部正式实验和后续的消融、稳定性、粒度和多轮对话实验。三个模型均通过第三方统一 API 网关接入, 切换模型只需修改一个环境变量。

表 4.4 实验模型与角色说明

模型名称	提供商	角色	实验覆盖
claude-haiku-4-5	Anthropic	较弱基线	VerilogEval 基线 + RTLLM 正式
claude-sonnet-4-6	Anthropic	中强模型	正式 + 消融 + 稳定性 + 粒度 + 多轮
gpt-5.4	OpenAI	强模型	正式 + 消融 + 稳定性 + 粒度 + 多轮 + 策略

除另有说明外, 所有实验使用以下统一参数: temperature=0.7, max_tokens=4096, 每轮候选数 $k = 3$, 最大迭代轮数为 5, 默认反馈级别为 L3 Succinct, 默认提示策略为 P0 Base。默认选择 L3 是实验设计阶段的先验设置, 因为在进行反馈粒度实验前尚无法



确定哪一级反馈最优；粒度实验的目的之一正是检验这一默认选择是否合理。

4.4 实验条件与递进逻辑

七组实验按照三个层次逐步深入，每个层次回答不同的问题，后一层次的实验建立在前一层次的结论之上。

4.4.1 第一层：有效性验证

在确认系统能够正确运行之后，首先需要回答的基本问题是：反馈循环是否有效？

VerilogEval-Human 基线实验使用 Haiku 模型在 20 题上运行零样本和 Feedback 两种条件，目的是验证反馈循环在基础难度任务上的效果，同时确认系统从代码生成到结果评估的完整流程能够正常工作。

RTLLM_STUDY_12 正式实验将验证扩展到更难的任务和更多的模型：在 12 题上对 Haiku、Sonnet 和 GPT-5.4 分别运行零样本 ($k=1$ ，不迭代) 和 Feedback ($k=3$ ，最多 5 轮，L3 反馈) 两种条件。通过三个模型的交叉对比，可以观察反馈效果是否具有跨模型一致性。

4.4.2 第二层：收益归因与稳定性

第一层实验如果显示反馈有效，随之而来的问题是：这个提升有多少来自反馈信息本身，有多少仅仅是因为多试了几次？单次实验的结论是否稳定？

消融实验设计了三个对照条件来分离这两个因素。零样本 (Zero-shot) 只有 1 次生成机会，不含反馈。Retry-only 拥有与 Feedback 相同的生成预算 ($k=3$ ，最多 5 轮，即最多 15 次)，但不注入任何错误反馈信息，每轮都使用初始提示词从头生成。Feedback 同样拥有 15 次生成预算且注入 L3 反馈。比较 Retry-only 与 Feedback 的差异，就能将反馈信息的独立贡献从多采样效应中分离出来。

但消融实验的结论可能受到 LLM 生成随机性的影响。为评估结论的统计稳健性，对 GPT-5.4 和 Sonnet 分别进行了 4 轮完全独立的重复实验，每轮使用相同参数但不同的随机种子。如果某个结论在 4 轮中一致成立，则可以认为它不是偶然现象。

4.4.3 第三层：机制探索

前两层确认了反馈的有效性和收益来源后，第三层探索具体的优化方向。



反馈粒度实验在 GPT-5.4 和 Sonnet 上运行 L0 至 L4 五级反馈条件, 研究反馈中应包含多少信息才能最大化修复效果。

多轮对话实验探索将反馈组织为连续对话 (Multi-turn) 而非独立请求 (Single-turn) 的效果差异。该实验在 6 题代表性子集上运行, 因 API 成本较高而限制了规模。需要说明的是, 该实验初始版本存在代码 bug, 本文报告的结果均来自修复后的 v2 版本。

提示策略实验在 GPT-5.4 上对比 Base、CoT[27]、Few-shot 和 Few-shot+CoT[10] 四种提示策略, 研究初始提示词质量与反馈循环效果的交互关系。

4.4.4 实验设计与第 5 章结果章节的对应关系

为便于读者将本章定义的三层实验设计与第 5 章的结果分析逐一对照, 表 4.5 列出了每一层、每一组实验在第 5 章的对应小节。第 5 章将严格按照这一对应关系组织: 5.1–5.2 对应有有效性验证, 5.3–5.4 对应收益归因与稳定性, 5.5–5.7 对应机制探索, 5.8–5.10 对典型案例和综合结论进行补充讨论。

表 4.5 实验设计与第 5 章结果章节的对应关系

实验层次	实验名称	设计目的	第 5 章对应
有效性验证	VerilogEval-Human 基线	验证反馈循环基础有效性	5.1
有效性验证	RTLLM_STUDY_12 正式	更难任务和多模型验证	5.2
收益归因	Feedback vs Retry-only 消融	分离反馈信息与多采样贡献	5.3
稳定性验证	4 轮重复实验	检查结论统计稳健性	5.4
机制探索	反馈粒度实验	分析反馈信息量影响	5.5
机制探索	多轮对话反馈实验	分析反馈组织方式影响	5.6
机制探索	提示策略实验	分析初始提示词影响	5.7
补充分析	典型案例分析	解释成功、失败与边界	5.8
补充分析	综合讨论与小结	跨实验整合与对照	5.9–5.10



4.5 评价指标与统计方法

通过率 (Pass Rate) 是首要评价指标, 定义为通过测试平台全部测试用例的任务数占总任务数的比例。Rank 分数作为辅助指标, 取值范围为 $[-2.0, 1.0]$ 。派生指标包括 Feedback 改善题数和 API 错误率。对于稳定性重复实验, 报告 4 轮重复的均值和标准差; 对于单次运行的实验, 在结论中注明其局限性。

4.6 实验可靠性控制

审计与验证机制包括代码审计 (4 项检查) 和数据审计 (覆盖全部 7 个实验类别), 两项均已通过。实验结果索引明确标注每个实验运行的状态, 确保论文中每一个实验数值均可追溯。

4.7 本章小结

本章定义了全部实验方案。实验按“有效性验证—收益归因—机制探索”三个层次递进展开, 涵盖两个评估基准、三个模型和七组实验。各实验与研究问题的对应关系参见表4.1。



5 实验结果与分析

本章按照第 4 章提出的三层实验设计展开结果分析：5.1–5.2 对应有有效性验证，5.3–5.4 对应收益归因与稳定性验证，5.5–5.7 对应机制探索，5.8–5.10 对典型案例和综合结论进行补充讨论。与表4.5的对应关系逐一对齐，所有数据以实验结果索引为准。

5.1 VerilogEval-Human 基线实验

首先报告第一层有效性验证实验。本节记录在 VerilogEval-Human 20 题子集上使用 Haiku 模型进行的基线实验，用以确认反馈循环在较低难度任务上的基本有效性。反馈循环将通过率从 80% 提升至 90%，具体结果如表5.1所示，反馈修复了零样本未通过 4 题中的 2 题。

表 5.1 VerilogEval-Human 20 题基线实验结果

条件	通过数	通过率
Zero-shot	16/20	80%
Feedback	18/20	90%
提升	+2	+10 个百分点

反馈成功修复的两道题 Prob109_fsm1 和 Prob127_lemmings1，均属于代码接近正确但存在局部状态转移错误的场景——反馈循环对这类“差一步”的问题尤为有效。未能修复的两道题分别涉及 VerilogEval-Human 中 LFSR 类题目的抽头多项式（精确数学知识）和 HDLC 协议边界条件（深层协议逻辑），表明反馈对领域知识缺口的修复能力有限。需要说明的是，此处的 LFSR 与 §5.2 中 RTLLM_STUDY_12 内的 LFSR 是不同基准下的不同设计，不应混淆。

5.2 RTLLM_STUDY_12 正式实验结果

在 RTLLM_STUDY_12 上使用三个模型进行零样本与 Feedback 对比,结果如表5.2所示。三组实验的 API 错误率均为零。

各题目的逐题结果详见附录 B 图B.1。

反馈对所有模型均产生了正向增益，但强模型从反馈中获益最大——GPT-5.4 的增



表 5.2 RTLLM_STUDY_12 三模型正式实验结果

模型	Zero-shot	Feedback	提升	改善题数
Haiku 4.5	5/12 (42%)	6/12 (50%)	+8pp	2
Sonnet 4.6	5/12 (42%)	7/12 (58%)	+17pp	2
GPT-5.4	6/12 (50%)	10/12 (83%)	+33pp	4

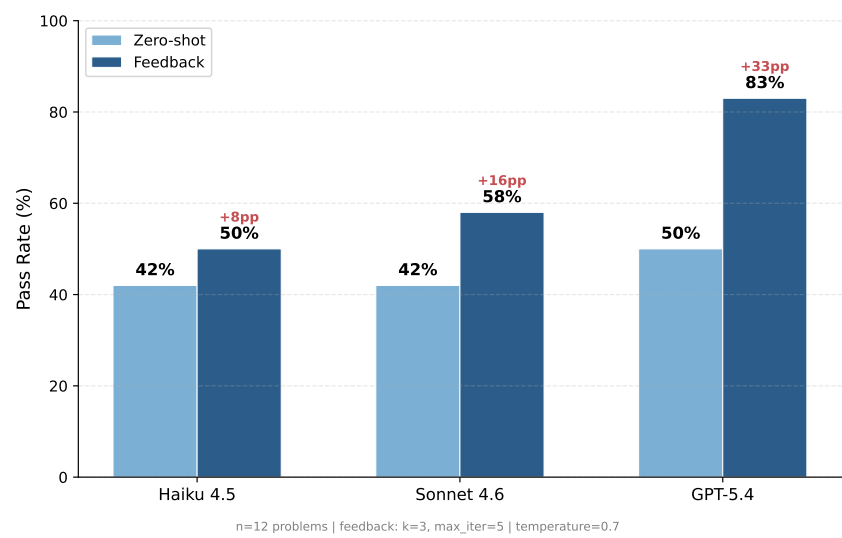


图 5.1 RTLLM_STUDY_12 三模型通过率对比



益 (+33 个百分点) 远超 Sonnet (+17 个百分点) 和 Haiku (+8 个百分点)。这一现象的可能解释是：反馈信息需要模型具备足够的代码理解和修复能力才能被有效利用。强模型在接收到编译或仿真错误信息后，更能准确定位问题根因并生成正确的修复方案；而弱模型即使获得了相同的错误信息，也可能因基础能力不足而无法将反馈转化为有效修复。由此推断，反馈循环的收益上限取决于模型自身的代码推理能力。

实验还发现了两类值得关注的特殊题目。第一类是零样本失败但反馈可修复的题目：LFSR 和 traffic_light 在零样本下所有模型均失败，但在反馈条件下被成功修复，说明这类题目的主要障碍不是设计复杂度而是初始生成中的局部错误，反馈能够精准定位并修复这些错误（注：本节的“零样本失败但反馈可修复”涵盖 retry-only 也可能解出的情况，与 §5.3 中“retry-only 反复重试均无法通过、仅反馈可解”是不同的概念，将在 §5.3 中具体区分）。第二类是天花板题：sequence_detector 和 freq_divbyfrac 在所有模型和条件下均未通过，暴露了当前反馈框架对涉及精确算法知识的设计任务的局限。

各模型反馈增益的详细对比见附录 B 图B.2。

5.3 Feedback vs Retry-only 消融实验

在确认反馈循环整体有效后，接下来进入第二层收益归因分析，需要回答的疑问是：这个提升究竟来自反馈信息本身，还是仅仅因为多采样机会增加？为区分反馈循环中“多次重试”与“错误反馈信息”的各自贡献，在 GPT-5.4 和 Sonnet 4.6 上运行了三条件消融实验（实验设计详见第 4 章）。设计 retry-only 条件的核心动机在于：反馈循环同时带来了两个优势——更多的生成机会和来自 EDA 工具的错误信息，如果不加以分离，就无法判断反馈的实际价值。

GPT-5.4 消融结果。 GPT-5.4 的单轮消融结果为：Zero-shot 50% (6/12)，Retry-only 67% (8/12)，Feedback 83% (10/12)。由此可见，从零样本到反馈的 +33 个百分点总提升中，多采样贡献 +17 个百分点，反馈信息贡献 +16 个百分点，反馈信息约占总提升的 49%。需要注意的是，单轮结果受随机波动影响，稳定性实验（4 轮均值，见 5.4 节）修正后的比例为反馈信息约占 57%。这一差异提示我们：对于涉及随机采样的实验，单轮数据应谨慎解读。

消融实验中较为直接的证据来自仅反馈可解的题目（即 retry-only 反复重试也无法通过、只有引入反馈信息后才能修复的题目）：sequence_detector 和 traffic_light 在 15 次独立重试中从未通过，但在反馈条件下于 2–3 轮内成功修复，说明错误反馈信息提供了

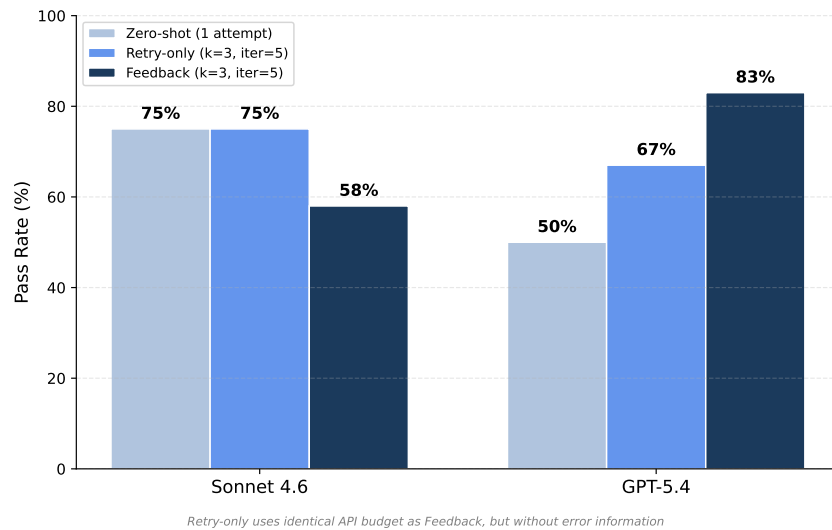
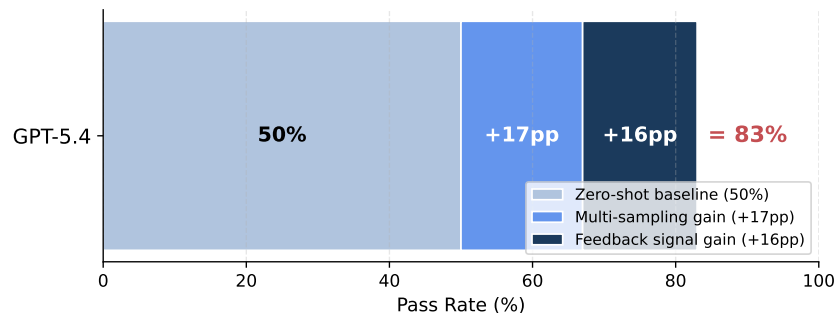


图 5.2 零样本/重试/反馈三条件对比

独立采样无法获得的修复信号。



sequence_detector & traffic_light: only solvable by feedback (15 retries all failed)

图 5.3 GPT-5.4 反馈收益来源分解（基于单轮消融实验数据，对应 §5.4 节的 4 轮稳定性实验给出修正后的 57% 占比）

Sonnet 4.6 消融结果。 Sonnet 4.6 的单轮消融结果显示 Feedback（58%）低于 Retry-only（75%）。这一结果表明：对 Sonnet 而言，在当前配置下，与其让模型依据反馈信息尝试修复，不如让其从头重新生成。可能的原因是 Sonnet 在处理反馈信息时倾向于对已正确部分进行过度修改，导致修复引入新的错误。该结论是否稳定，需结合下一节的稳定性实验综合判断。

5.4 稳定性重复实验

消融实验的结论是否具有统计可靠性？单轮实验受 LLM 生成随机性影响较大，因此本节通过 4 轮独立重复来检验核心结论的稳定性。每轮使用相同的参数配置但不同的随机种子，产生完全独立的实验结果。

GPT-5.4。 4 轮独立重复中，Feedback 始终优于 Retry-only (4/4 轮)，且无一例外。反馈不仅提升了平均通过率，还将标准差从 11.8%（零样本）压缩至 4.2%，表明反馈循环不仅提高了成功率，还降低了结果的波动性，如表5.3所示。

表 5.3 GPT-5.4 稳定性实验统计

统计量	Zero-shot	Retry-only	Feedback
均值	50.0%	62.5%	79.2%
标准差	$\pm 11.8\%$	$\pm 4.2\%$	$\pm 4.2\%$
FB>RO 轮次	—	—	4/4

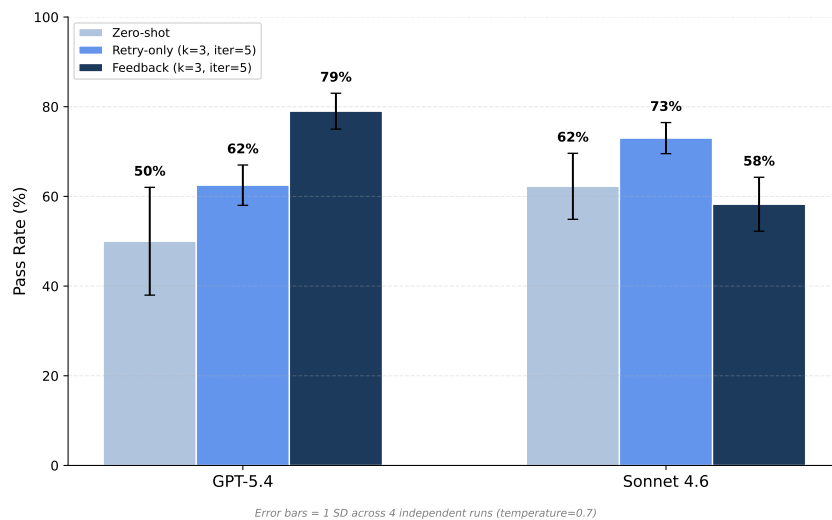


图 5.4 稳定性实验三条件均值与标准差

Sonnet 4.6。 Sonnet 上 Feedback（均值 58.3% $\pm$ 5.9%）低于 Retry-only（72.9% $\pm$ 3.6%）的结论在 4 轮中均成立（0/4 轮 FB 优于 RO），如表5.4所示。

模型依赖性。 两个模型对反馈的响应截然相反：GPT-5.4 上 FB 相对 RO 的增益为 +16.7 个百分点，Sonnet 上为 -14.6 个百分点。这一分化在 4 轮重复中完全稳定（标准差均小于 6%），说明这不是随机噪声而是两个模型在利用反馈信息方面的本质差异。这一发现

表 5.4 Sonnet 4.6 稳定性实验统计

统计量	Zero-shot	Retry-only	Feedback
均值	62.5%	72.9%	58.3%
标准差	$\pm 7.2\%$	$\pm 3.6\%$	$\pm 5.9\%$
FB>RO 轮次	—	—	0/4

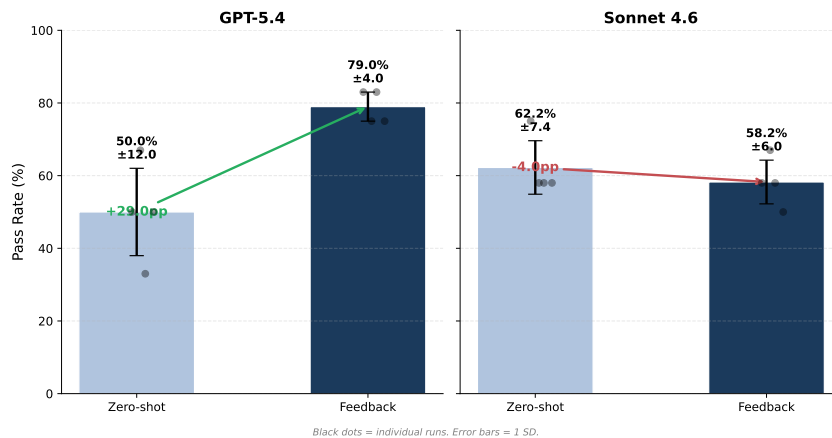


图 5.5 正式实验重复结果对比

对实际部署有参考价值：在选择是否启用反馈循环之前，应先在目标模型上进行小规模验证。

反馈信息独立贡献的稳定估计。基于 GPT-5.4 的 4 轮稳定性均值，§5.3 给出的”反馈信息约占总提升 57%”可显式写为：反馈信息独立贡献为 $(79.2\% - 62.5\%) / (79.2\% - 50.0\%) \approx 57.2\%$ ，其中分子是 Feedback 相对 Retry-only 的均值增益（反馈信息的独立效应），分母是 Feedback 相对 Zero-shot 的均值总增益。该比例与 §5.3 单轮消融估计的 49% 在数量级上一致，但稳定性实验下噪声更小，可作为本文消融分析的主参考值。

5.5 反馈粒度实验

在完成有效性和收益归因分析后，本节开始进入第三层机制探索，首先考察反馈信息量的影响。反馈中应包含多少信息是一个开放问题：信息越多固然为模型提供了越完整的错误图景，但过多的信息也可能干扰模型对关键错误的定位。本节通过五级粒度实验（L0–L4）系统测试信息量对修复效果的影响。需要说明的是，本节粒度实验为单次运行，与 §5.4 节稳定性实验所用的随机种子不同，因此 L3 条件下的取值（GPT-5.4：92%，Sonnet 4.6：67%）与 §5.4 节的 4 轮均值（GPT-5.4：79.2%，Sonnet 4.6：58.3%）在单次

结果上存在自然波动，应在波动范围内整体看待两节数据。结果如表5.5所示，反馈粒度存在较明显的最佳区间：通过率随反馈信息量的增加先上升，在 L2（Compile-only）和 L3（Succinct）附近达到较好水平，随后在过量反馈条件下可能下降。GPT-5.4 的曲线呈现较明显的倒 U 型趋势，Sonnet 4.6 则在 L2 后回落并趋于平稳。

表 5.5 反馈粒度实验通过率

级别	GPT-5.4	Sonnet 4.6
L0 Zero-shot	58% (7/12)	50% (6/12)
L1 Retry-only	75% (9/12)	67% (8/12)
L2 Compile-only	92% (11/12)	75% (9/12)
L3 Succinct	92% (11/12)	67% (8/12)
L4 Rich	83% (10/12)	67% (8/12)

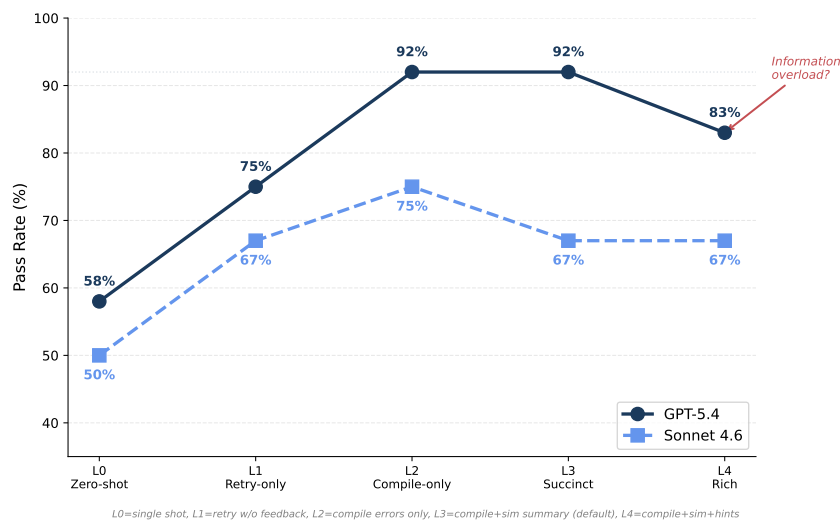


图 5.6 反馈粒度通过率曲线

从实践角度看，L1→L2 是最大的跃升点（GPT-5.4 上从 75% 跃升至 92%），表明即使是最基础的编译错误反馈也具有极高的修复价值。L2 Compile-only 在两个模型上均为最优或并列最优，是较稳健的通用选择。L4 Rich 反馈包含多达 80 行仿真输出和分析提示，其通过率反而低于 L2 和 L3，可能的原因是：过于冗长的仿真日志稀释了关键错误信息的信噪比，使模型难以从中提取修复所需的核心线索。这一“信息过载”现象与注意力机制在长上下文中常见的信噪比下降问题方向一致：当反馈中非关键信息超过一定比例时，模型对关键错误位置的定位反而被稀释。



各题目在不同粒度下的逐题结果详见附录 B 图B.3。

5.6 多轮对话反馈实验

前述实验均采用单轮 API 调用模式，即每次反馈都发起一次独立请求。本节探索将反馈组织为多轮对话（Multi-turn），使模型能够看到完整的修复历史。本节结果均来自 v2 修正版。

GPT-5.4 和 Sonnet 4.6 的多轮对话实验结果分别如表5.6和表5.7所示。在控制候选数量的公平对比下（ST $k=1$ vs MT $k=1$ ），GPT-5.4 从 83% 提升至 100%、Sonnet 4.6 从 33% 提升至 50%，两个模型的 +1 题改善方向相同（在 6 题子集上分别等价于 +17 个百分点）。这一结果表明，持续的对话上下文有助于模型追踪修复历史和错误演变模式，避免在不同轮次中重复相同的错误路径。需要注意的是，MT $k=1$ 与 ST $k=3$ 的比较并不公平（候选数量不同），仅可作为参考。此外，该实验在 6 题子集上进行且为单次运行，6 题子集和 +1 题改善的样本量均较小，统计效力有限，结论需谨慎推广。

表 5.6 多轮对话反馈 v2 实验结果（GPT-5.4）

条件	通过率
A: ST $k=3$	83% (5/6)
D: ST $k=1$	83% (5/6)
B: MT $k=1$	100% (5/5)*
C: CO $k=3$	83% (5/6)

*freq_divbyfrac 因 API 超时排除。

表 5.7 多轮对话反馈 v2 实验结果（Sonnet 4.6）

条件	通过率
A: ST $k=3$	17% (1/6)
D: ST $k=1$	33% (2/6)
B: MT $k=1$	50% (3/6)
C: CO $k=3$	33% (2/6)

多轮对话实验的逐题结果详见附录 B 图B.4。

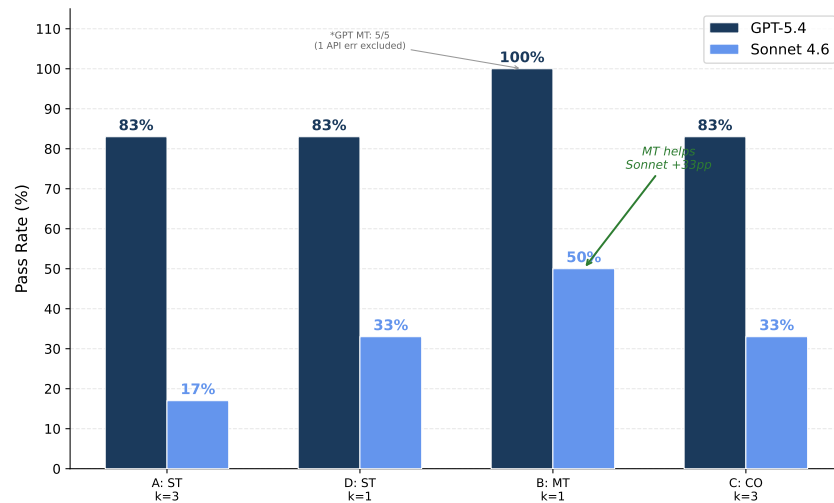


图 5.7 多轮对话反馈 v2 条件对比（公平对比对为 ST $k=1$ 与 MT $k=1$ ，候选数相同；ST $k=3$ 和 CO $k=3$ 列因候选数不同仅作参考）

5.7 提示策略实验

初始提示词的质量是否影响最终代码正确率？本节在 GPT-5.4 上对比四种提示策略（Base、CoT、Few-shot、Few-shot+CoT），分析提示词工程与反馈循环的交互关系。结果如表5.8所示。

表 5.8 提示策略实验结果（GPT-5.4）

策略	Zero-shot	Feedback	FB 增益
P0: Base	58% (7/12)	92% (11/12)	+34pp
P1: CoT	50% (6/12)	92% (11/12)	+42pp
P2: Few-shot	67% (8/12)	92% (11/12)	+25pp
P3: Few-shot+CoT	58% (7/12)	92% (11/12)	+34pp

在零样本阶段，Few-shot（P2）以 67% 成为最优策略，说明提供具体示例有助于模型理解 Verilog 代码结构。CoT（P1）反而略低于基线，可能因为链式推理在 RTL 生成任务中增加了不必要的中间步骤，引入了额外的出错机会。

然而引入反馈后，四种策略全部收敛至 92%（11/12），仅 `freq_divbyfrac` 在所有策略下均未通过。这一“拉平效应”说明，反馈循环是决定最终代码质量的主导因素，初始提示策略更多影响零样本起点而非经过反馈修复后的最终上限。从实践角度看，这意味着在配备了反馈循环的系统中，精细的提示词工程的边际收益有限，系统设计者可以将

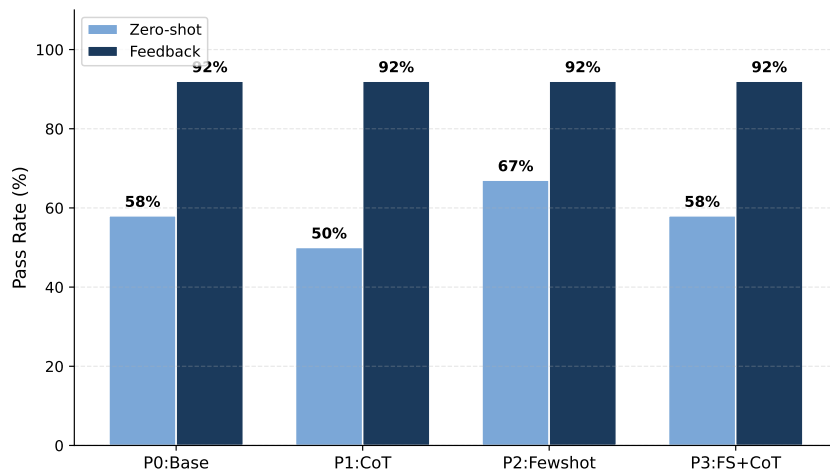


图 5.8 提示策略零样本与反馈对比

更多精力投入到反馈信息的构建上。

各策略的逐题结果详见附录 B 图B.5。

5.8 典型案例分析

前述结果给出了三层实验的整体统计结论，本节选取典型题目进一步解释反馈循环在具体任务上的作用方式，覆盖反馈修复成功、反馈弥补模型差距、反馈稳定收益与天花板题四类场景。

float_multi：反馈修复最高难度设计

float_multi（IEEE 754 浮点乘法器）是 STUDY_12 中难度最高的题目，要求实现符号位处理、指数加法、尾数乘法和规格化等多个步骤。三个模型在零样本下全部失败，典型的错误模式包括指数偏移计算错误和尾数乘积截断方式不正确。然而，Sonnet 和 GPT-5.4 均在反馈的第 1 轮迭代中成功修复——编译错误信息准确指出了语法问题，仿真反馈则帮助模型定位了具体的数值计算偏差。Haiku 即使有反馈仍无法修复，表明该题的修复需要模型具备一定的浮点运算先验知识，反馈信息本身不足以弥补基础能力的差距。

LIFObuffer：反馈弥补模型差距

LIFObuffer（后进先出缓冲区）展示了反馈机制的另一种价值。GPT-5.4 零样本即通过，说明该题对强模型不构成挑战。但 Haiku 在零样本下生成的代码存在读写指针管理



错误，反馈循环在第 1 轮提供了仿真输出中的指针越界信息，Haiku 据此成功修复，追平了 GPT-5.4 的零样本水平。这一案例说明反馈机制在资源受限场景下提供了“以计算换准确度”的可行路径——使用较弱但成本更低的模型配合反馈循环，可以达到与强模型零样本相当的效果。

adder_bcd 与 div_16bit：反馈的稳定收益

adder_bcd（BCD 加法器）和 div_16bit（16 位除法器）是两道中等难度的算术题目。在正式实验中，这两道题在多个模型上呈现出一致的规律：零样本下编译通过但仿真部分失败，反馈 1–2 轮后成功修复。仿真反馈中的不匹配样本数为模型提供了明确的修复方向，这类“功能接近正确但存在边界错误”的场景是反馈循环最典型的应用场景。

天花板题分析

sequence_detector 在正式实验中所有模型和条件均编译失败，freq_divbyfrac 在所有条件下均未通过仿真。对 sequence_detector 的错误日志分析表明，模型生成的代码在模块接口上与测试平台不匹配，且多轮反馈未能纠正这一结构性问题。然而，sequence_detector 在后续的消融稳定性实验和提示策略实验中被成功修复，说明该题并非绝对不可解，而是对初始生成路径高度敏感——不同的随机种子或提示词可能导致截然不同的初始代码结构。freq_divbyfrac（分数分频器）则涉及精确的小数分频算法，属于当前框架难以仅通过反馈解决的领域知识缺口问题。

5.9 综合讨论

综合七组实验的结果，主要结论是：EDA 工具反馈能够提升大语言模型生成 Verilog 代码的正确率，但这一提升并非对所有模型普适有效。GPT-5.4 在 RTLLM_STUDY_12 上获得了 +33 个百分点的增益，且在 4 轮重复中稳定存在（79.2%±4.2%）；而 Sonnet 4.6 上反馈反而导致 -14.6 个百分点的回退，同样在 4 轮中完全稳定。

消融实验揭示了反馈收益的双重来源：约 43% 来自多次采样机会，约 57% 来自错误反馈信息本身。后者的独立价值通过仅反馈可解的题目得到了直接验证——这些题目在 15 次独立重试中从未通过，但在 2–3 轮反馈后成功修复。



Sonnet 反馈退化的具体表现

对 Sonnet 4.6 的 4 轮消融实验逐题分析发现, 反馈退化集中在特定题目上。div_16bit 在 4 轮中 Retry-only 全部通过 (4/4), 但 Feedback 只通过了 1 轮 (1/4)——这意味着反馈信息反而干扰了 Sonnet 对该题的修复。LIFObuffer 呈现类似的模式: Retry-only 通过了 2 轮 (2/4), Feedback 通过了 0 轮 (0/4)。这两道题目的共同特征是涉及较复杂的数据路径操作 (16 位除法和读写指针管理), Sonnet 在零样本或重试中能够偶尔生成正确实现, 但在接收到 L3 反馈信息后倾向于过度修改已接近正确的代码结构。

结合粒度实验的数据, Sonnet 在 L2 (仅编译反馈) 条件下的通过率为 75%, 高于 L1 Retry-only 的 67%, 而 L3 和 L4 均回落至 67%。这一对比表明: Sonnet 并非完全无法利用反馈, 而是对反馈信息量敏感——编译错误这类简短、明确的反馈对 Sonnet 仍有帮助, 但包含仿真输出摘要的 L3 反馈可能引入了额外的噪声, 导致模型偏离已接近正确的修复方向。这一发现对实际部署有直接启示: 在使用中等能力模型时, 应优先尝试 L2 反馈而非默认使用 L3。

成本与收益的权衡

反馈循环需要比零样本更多的 API 调用。零样本每题 1 次调用, 反馈条件下每题最多 $k \times N = 3 \times 5 = 15$ 次调用。Retry-only 的调用上限与 Feedback 相同。从已有实验记录来看, 在 GPT-5.4 的正式实验中, 10 道通过题中有 8 道在第 1 轮 (3 次调用) 内收敛, 仅 2 道需要多轮迭代。4 轮重复实验的统计显示, GPT-5.4 的 Feedback 平均迭代轮数为 2.0 轮 (平均约 6 次调用), 远低于 15 次的理论上限。

从成本效益角度看, 反馈循环的价值在高难度任务上最大: GPT-5.4 在 RTLLM_STUDY_12 上从 50% 提升到 83%, 额外的 API 调用换来了 +33 个百分点的通过率增益。但对于零样本通过率已较高的简单任务 (如 VerilogEval-Human 的 80% 基线), 反馈的边际收益仅为 +10 个百分点, 此时额外成本的性价比较低。一个更经济的策略是: 先对全部题目运行零样本, 仅对未通过的题目启用反馈循环。

在粒度和策略方面, 编译反馈 (L2) 是最稳健的通用选择。反馈信息量过多时可能干扰模型对关键错误的定位, 反而不如简洁的编译错误信息有效。多轮对话和提示策略可作为补充优化方向。



与已有工作的关系

本文的反馈循环机制受到 AutoChip[5] 等已有工作的启发。与 AutoChip 侧重于验证反馈循环的可行性不同, 本文更关注该机制在更难基准、不同能力模型和不同反馈策略下的表现。需要说明的是, 由于本文使用的模型 (Claude-Haiku-4-5、Sonnet-4-6、GPT-5.4) 与 AutoChip 原文 (GPT-3.5 等) 在发布时间和能力等级上差异较大, 且评估子集与提示词模板不完全一致, 直接的数值对比意义有限, 表5.9因此仅从六个维度做定性对照, 总结本文工作相对已有 feedback-in-the-loop 思路的扩展。

表 5.9 本文系统与已有反馈循环工作的对比

维度	已有工作侧重	本文工作
任务基准	可行性验证	VerilogEval + RTLLM_STUDY_12
模型覆盖	单/少量模型	Haiku / Sonnet 4.6 / GPT-5.4
反馈收益归因	关注总体提升	Zero-shot / Retry-only / Feedback 消融
反馈策略	单一反馈模式	L0-L4 五级粒度 + 多轮对话
结论可靠性	较少重复验证	4 轮独立重复实验
实验管理	结果记录	元数据 + 结果索引 + 审计脚本

因此, 本文并非停留在对已有反馈循环思路的复现, 而是在实验规模、对照条件和结果可追溯性方面做出了扩展。

表5.10汇总了各实验的主要结论。

表 5.10 主要实验结论汇总

实验	关键对比	主要结论
基线实验	Zero-shot vs FB	通过率 80%→90%
正式实验	三模型对比	GPT-5.4 增益最大 (+33pp)
消融实验	FB vs Retry-only	反馈信息贡献 ~57%
稳定性	4 轮重复	GPT-5.4 结论稳定 (4/4)
粒度实验	L0-L4	反馈粒度存在最佳区间, L2 较稳健
多轮对话	ST vs MT	MT 有优势, 需扩大验证
策略实验	P0-P3	反馈拉平至 92%



局限性

RTLLM_STUDY_12 为 12 题子集，虽然覆盖了四大类别和多个难度梯度，但不能完全代表所有类型的 RTL 设计任务。部分实验（多轮对话、提示策略）为单次运行或在小子集上进行，统计效力有限。Sonnet 上反馈无效的深层原因尚未完全明确，可能需要更细粒度的错误分析来回答。天花板题暴露了当前框架对涉及精确算法知识的设计任务的局限。此外，实验仅使用 Icarus Verilog 作为仿真工具，商业 EDA 工具可能提供更丰富的诊断信息。

5.10 本章小结

本章按照第 4 章提出的三层实验设计展开。三层实验共同支持了一个主结论：反馈循环对足够强的模型（如 GPT-5.4）和足够难的任务（RTLLM_STUDY_12）效益最大，但其有效性同时受模型能力限制（Sonnet 4.6 上反馈反而退化）和反馈信息量限制（L4 Rich 反馈不如 L2 Compile-only）。具体而言，有效性验证层（对应 5.1–5.2）支持了反馈循环在不同难度和模型上能够稳定地提升通过率；收益归因与稳定性层（对应 5.3–5.4）通过消融与 4 轮重复实验，进一步把这一提升中可归因于反馈信息本身的部分稳定估计为约 57%，并把 Sonnet 的退化定性为不同模型对反馈利用方式的本质差异而非随机噪声；机制探索层（对应 5.5–5.7）则在主结论之上给出了三个局部优化方向——L2 Compile-only 是较稳健的粒度起点，多轮对话在小样本上呈现出与单轮反馈方向相同的改善（受样本量限制需更大规模实验验证），提示策略主要影响零样本起点而反馈后被拉平至同一水平。在此基础上，典型案例与综合讨论（对应 5.8–5.9）进一步给出了“高难度任务优先启用反馈循环、中等能力模型优先尝试 L2 粒度、简单任务可仅对未通过题目启用反馈”等可参考的部署建议。总体看，本文工作在实验规模、消融归因、稳定性验证和反馈粒度探索上对已有反馈循环工作进行了系统性扩展。



6 总结与展望

6.1 本文工作总结

本文构建了一个基于 EDA 工具反馈的 LLM Verilog 生成与自动优化原型系统，并完成了系统实现和实验验证两部分工作。

系统以 Icarus Verilog 为编译仿真后端，实现了“生成—编译—仿真—反馈—修复”反馈循环，支持 VerilogEval 和 RTLLM 双 benchmark 适配、多模型统一调用、可配置的反馈粒度和提示策略。围绕核心系统，还完成了 RTLLM 兼容性扫描（50 题中 41 题完全可用）、RTLLM_STUDY_12 正式研究子集的选取、多模式实验运行脚本和结果图表整理等配套工作。

在实验方面，设计并执行了七组递进式实验，涵盖基线验证、正式对比、消融分析、稳定性验证、反馈粒度、多轮对话和提示策略等维度。

6.2 主要实验结论

基于七组实验的结果，本文的核心发现可以概括为以下几点。

EDA 工具反馈能够提升 RTL 代码生成正确率。在 VerilogEval-Human 上，Haiku 模型的通过率从 80% 提升至 90%；在 RTLLM_STUDY_12 上，GPT-5.4 从 50% 提升至 83%，绝对提升达 33 个百分点。反馈的收益并非简单的多次重试：消融实验表明反馈信息本身贡献了约 57% 的提升，且存在仅反馈可解而 15 次独立重试均无法通过的题目。

反馈效果具有模型依赖性，GPT-5.4 上反馈始终优于 Retry-only（4/4 轮），而 Sonnet 4.6 上反馈在当前设置下反而不如 Retry-only（0/4 轮）。反馈信息量存在最佳区间，粒度实验显示编译反馈和简洁反馈较为稳健，且 GPT-5.4 上呈现较明显的倒 U 型趋势。多轮对话反馈和提示策略可作为补充优化方向。

6.3 系统局限性

本文的实验和系统存在以下局限。RTLLM_STUDY_12 包含 12 道精选题目，虽然覆盖四大类别和较高难度梯度，但不能代表所有类型的 RTL 设计任务。正式实验集中在三个模型上，其他模型的反馈响应模式尚未验证。当前系统的反馈信息主要基于仿真文本输出，缺少更精确的波形级定位。部分实验为局部探索，样本量较小，统计效力有



限。

6.4 后续工作展望

当前工作留下了若干值得深入研究的方向。

在基准覆盖方面，可将实验范围从 `RTLSTM_STUDY_12` 扩展到更大的任务子集乃至全量 `RTLSTM`，并引入更多高难度 RTL 生成基准以验证结论的普适性。当前 12 题子集虽然覆盖了四大类别，但对某些特定领域（如通信协议、DSP 处理链）的覆盖不足，扩大任务范围有助于发现反馈机制在不同设计领域的差异化表现。

在模型覆盖方面，可纳入更多闭源和开源大语言模型进行对照。当前实验发现的模型依赖性现象（GPT-5.4 上反馈有效而 Sonnet 上无效）提示不同模型对反馈信息的处理机制可能存在本质差异。通过扩大模型覆盖范围，有望总结出哪些模型特性有利于反馈利用，并据此探索根据模型特性和错误类型动态选择反馈策略的自适应机制。

在反馈精度方面，当前系统的仿真反馈主要基于文本级的仿真输出摘要，未能利用波形级的信号差异信息。后续可引入信号级差异分析，将仿真反馈从“第 N 个样本不匹配”提升到“信号 X 在时刻 T 的期望值与实际值不一致”的精度，这种更精确的反馈有望进一步提升修复效率。

在知识增强方面，天花板题（如 `freq_divbyfrac`）暴露了当前框架的一个根本局限：当模型缺乏特定算法的先验知识时，仅靠反馈信息难以引导模型发现正确的实现方案。针对这一问题，可探索通过检索增强 [7]——在修复过程中检索相似的 Verilog 设计实现作为参考——来补充模型的领域知识。

在验证方法方面，可探索将形式化验证工具集成到反馈循环中。形式化工具能够提供反例（counterexample），即导致设计行为不符合规格的具体输入序列，这些反例可作为比仿真输出更精确的修复线索。

6.5 本章小结

本章总结了本文在系统实现和实验分析方面完成的工作，概括了主要结论，讨论了系统和实验的局限性，并从基准扩展、模型覆盖、反馈精细化、知识增强和形式化验证等方面展望了后续研究。



结论

本文构建了一个基于 EDA 工具反馈的 LLM Verilog 生成与自动优化原型系统，并通过七组递进式实验验证了反馈循环的有效性和边界条件。

实验数据本身的具体取值已在摘要和第 5 章给出，此处不再重复，结论的总体含义在于：反馈循环带来的修复增益既能在 4 轮重复中稳定再现（GPT-5.4），也能在另一个能力相当的模型上完全不出现（Sonnet 4.6），并且这种分化既与反馈信息量的设置有关，也与模型在已有信息上做局部修复的倾向有关。一个值得后续关注的现象是：反馈循环的实际收益与模型对反馈信息的利用方式强耦合。这意味着未来反馈式 RTL 生成系统的提升方向，可能不再单纯地“提供更多反馈信息”，而是让模型本身更善于在已有的少量关键信号上做局部修复——这与近期 agent 训练领域强调“interactive scaling”的方向相吻合。

基于实验结果，有三条对实际部署有参考价值的建议。第一，在目标模型上部署反馈循环前，应先用少量题目做小规模验证，因为 Sonnet 的结果表明反馈并非对所有模型都有正向效果。第二，反馈粒度建议从 L2（仅编译错误）或 L3（简洁反馈）开始，过于详尽的反馈反而可能干扰模型的修复判断。第三，对于零样本通过率已较高的简单任务，反馈循环带来的额外收益可能不抵 API 调用成本的增加；更合理的策略是先尝试零样本生成，仅对失败的题目启用反馈。

本文的工作为理解 EDA 工具反馈在 LLM RTL 自动生成中的作用提供了实证分析和工程参考。



致谢

毕业设计即将画上句号，回想从选题到实验再到论文定稿的这大半年，心里感慨不少。

最想感谢的是我的指导教师王锐老师。选题阶段王老师帮我理清了“大语言模型 + 硬件设计”这个方向的切入点，中期答辩时一针见血地指出应当引入更强的模型和更难基准来验证结论——正是这条建议让我后来转向 RTLLM 基准并设计了多模型交叉实验，整篇论文的实验框架也由此成型。论文修改阶段王老师在立论、结构和表述上给出了许多具体而中肯的意见，每次讨论都让我受益很大。在此向王老师致以最诚挚的谢意。

感谢北京航空航天大学计算机学院四年来提供的学习平台和资源，让我在本科阶段就有机会接触前沿研究并完成一份相对完整的实验性工作。

感谢一起度过大学四年的同学和朋友们。课上课下的交流讨论、实验室里互相帮忙调试的日子，是这段时光里最珍贵的部分。

最后，感谢我的家人。是你们一直以来的理解、包容和支持，让我能够安心投入学业。无论未来走向哪里，家永远是最坚实的后盾。



参考文献

- [1] CHANG K, WANG Y, REN H, et al. ChipGPT: How far are we from natural language hardware design[J]. arXiv preprint arXiv:2305.14019, 2023.
- [2] THAKUR S, AHMAD B, et al. Verigen: A large language model for Verilog code generation[C]//Proceedings of the Design Automation and Test in Europe Conference (DATE). Antwerp, Belgium: IEEE, 2023.
- [3] LIU M, PINCKNEY N, KHAILANY B, et al. VerilogEval: Evaluating large language models for Verilog code generation[C]//Proceedings of IEEE/ACM International Conference on Computer-Aided Design (ICCAD). San Francisco, CA: IEEE, 2023.
- [4] LU Y, LIU S, ZHANG Q, et al. RTLLM: An open-source benchmark for design RTL generation with large language model[C]//Proceedings of the Asia and South Pacific Design Automation Conference (ASP-DAC). Incheon, Korea: IEEE, 2024.
- [5] THAKUR S, AHMAD B, FAN Z, et al. AutoChip: Automating HDL generation using LLM feedback[C]//Proceedings of the ML for Systems Workshop at NeurIPS. New Orleans, LA: [s.n.], 2023.
- [6] TSAI Y D, LIU M, REN H. RTLFixer: Automatically fixing RTL syntax errors with large language models[J]. arXiv preprint arXiv:2311.16543, 2024.
- [7] YAO X, et al. HDLdebugger: Streamlining HDL debugging with large language models [J]. arXiv preprint arXiv:2403.11671, 2025.
- [8] IEEE. IEEE standard for Verilog hardware description language: IEEE Std 1364-2005[S]. New York, NY: IEEE, 2006.
- [9] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems (NeurIPS). Red Hook, NY: Curran Associates, Inc., 2017.
- [10] BROWN T B, MANN B, RYDER N, et al. Language models are few-shot learners[C]//Advances in Neural Information Processing Systems (NeurIPS). Red Hook, NY: Curran Associates, Inc., 2020.
- [11] OUYANG L, WU J, JIANG X, et al. Training language models to follow instructions with



- human feedback[C]//Advances in Neural Information Processing Systems (NeurIPS). Red Hook, NY: Curran Associates, Inc., 2022.
- [12] CHEN M, TWOREK J, JUN H, et al. Evaluating large language models trained on code [J]. arXiv preprint arXiv:2107.03374, 2021.
- [13] ROZIÈRE B, GEHRING J, GLOECKLE F, et al. Code llama: Open foundation models for code[J]. arXiv preprint arXiv:2308.12950, 2023.
- [14] WILLIAMS S. Icarus Verilog[EB/OL]. 2024. <https://github.com/steveicarus/iverilog>.
- [15] WOLF C, GLASER J. Yosys — a free Verilog synthesis suite[C]//Proceedings of the Austrochip Workshop. Linz, Austria: [s.n.], 2013.
- [16] HDLBits. HDLBits — Verilog practice[EB/OL]. 2024. <https://hdlbits.01xz.net/>.
- [17] PINCKNEY N, LIU M, KHAILANY B, et al. Revisiting VerilogEval: Newer LLMs, in-context learning, and specification-to-RTL tasks[J]. arXiv preprint arXiv:2408.11053, 2024.
- [18] LIU S, FANG W, LU Y, et al. RTLCoder: Fully open-source and efficient LLM-assisted RTL code generation technique[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2024.
- [19] AUSTIN J, ODENA A, NYE M, et al. Program synthesis with large language models[J]. arXiv preprint arXiv:2108.07732, 2021.
- [20] LI Y, CHOI D, CHUNG J, et al. Competition-level code generation with AlphaCode[J]. Science, 2022, 378(6624):1092-1097.
- [21] MENZA M U I, HASSOUN S. AIvriL: Ai-driven RTL generation with verification-in-the-loop[J]. arXiv preprint arXiv:2409.11411, 2024.
- [22] MADAAN A, TANDON N, GUPTA P, et al. Self-refine: Iterative refinement with self-feedback[C]//Advances in Neural Information Processing Systems (NeurIPS). New Orleans, LA: Curran Associates, 2023.
- [23] SHINN N, CASSANO F, GOPINATH A, et al. Reflexion: Language agents with verbal reinforcement learning[C]//Advances in Neural Information Processing Systems (NeurIPS). New Orleans, LA: Curran Associates, 2023.
- [24] HO C T, KUO C L, KHAILANY B, et al. VerilogCoder: Autonomous Verilog coding agents with graph-based planning and abstract syntax tree (AST) tools[J]. arXiv preprint



arXiv:2408.08927, 2024.

- [25] BLOCKLOVE J, GARG S, KARRI R, et al. Chip-chat: Challenges and opportunities in conversational hardware design[C]//Proceedings of the ML for Systems Workshop at NeurIPS. New Orleans, LA: [s.n.], 2023.
- [26] FAN R, CHEN Y, ZHAO J, et al. AutoBench: Automatic testbench generation and evaluation using LLMs for HDL design[C]//Proceedings of the ML for Systems Workshop at NeurIPS. Vancouver, Canada: [s.n.], 2024.
- [27] WEI J, WANG X, SCHUURMANS D, et al. Chain-of-thought prompting elicits reasoning in large language models[C]//Advances in Neural Information Processing Systems (NeurIPS). Red Hook, NY: Curran Associates, Inc., 2022.



附录 A 实验配置与运行命令

本附录记录了本文实验使用的典型运行命令模板和关键配置。

A.1 正式实验运行命令

Zero-shot

```
python scripts/run_rtllm_subset.py \  
--model gpt-5.4 --subset study12 \  
--mode zero-shot
```

Feedback (default: k=3, iter=5, succinct)

```
python scripts/run_rtllm_subset.py \  
--model gpt-5.4 --subset study12 \  
--mode feedback
```

Ablation (zero-shot + retry-only + feedback)

```
python scripts/run_rtllm_subset.py \  
--model gpt-5.4 --subset study12 \  
--mode ablation
```

Granularity (L0-L4)

```
python scripts/run_rtllm_subset.py \  
--model gpt-5.4 --subset study12 \  
--mode granularity
```

Multiturn (ST/MT/CO conditions)

```
python scripts/run_rtllm_subset.py \  
--model gpt-5.4 --subset study12 \  
--mode multiturn
```



```
# Prompt strategy (P0-P3, ZS+FB each)
python scripts/run_rtlml_subset.py \
    --model gpt-5.4 --subset study12 \
    --mode prompt_strategy
```

A.2 环境配置

实验环境：Python 3.11，Icarus Verilog 12.0，Ubuntu 22.04。API 通过第三方统一网关接入。

A.3 核心代码模块索引

表 A.1 系统核心代码模块索引

模块	文件路径
反馈循环控制器	src/feedback/loop_runner.py
提示词构建器	src/feedback/prompt_builder.py
Verilog 执行器	src/runner/verilog_executor.py
评分器	src/ranking/ranker.py
RTLLM 加载器	src/runner/rtllm_loader.py
VerilogEval 加载器	src/runner/verilogeval_loader.py
LLM 客户端	src/llm/client.py
Verilog 提取器	src/utis/extract_verilog.py
实验产物管理	src/utis/artifacts.py
正式实验运行脚本	scripts/run_rtlml_subset.py
RTLLM 兼容性扫描	scripts/scan_rtlml_iverilog.py
代码审计	scripts/audit_project.py
数据审计	scripts/audit_data.py

附录 B 补充实验图表

以下图表为第 5 章实验结果的逐题细节补充, 供需要深入了解各题目表现差异的读者参考。

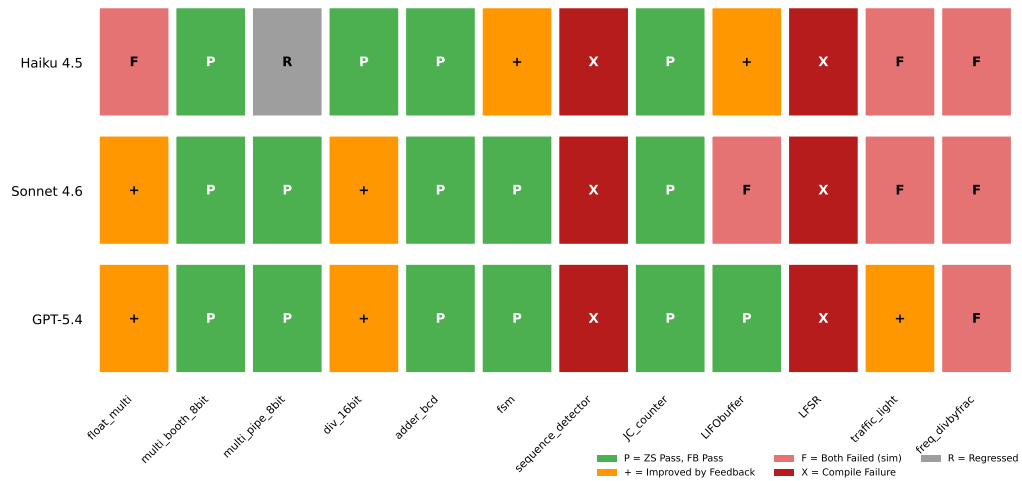


图 B.1 RTLLM_STUDY_12 逐题结果矩阵

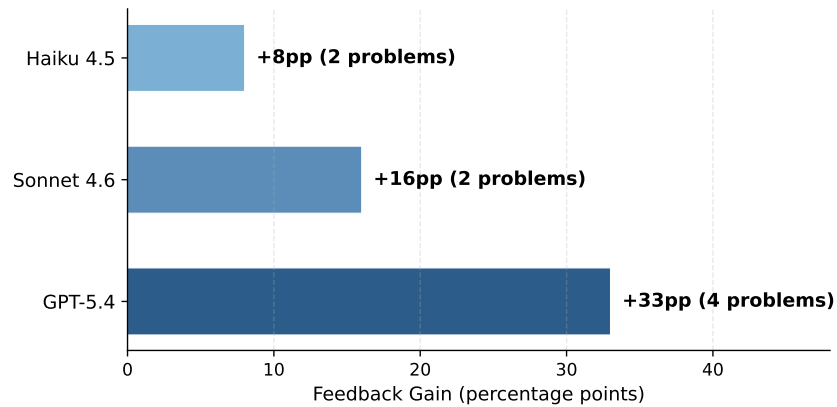


图 B.2 各模型 Feedback 增益对比



图 B.3 反馈粒度逐题结果矩阵

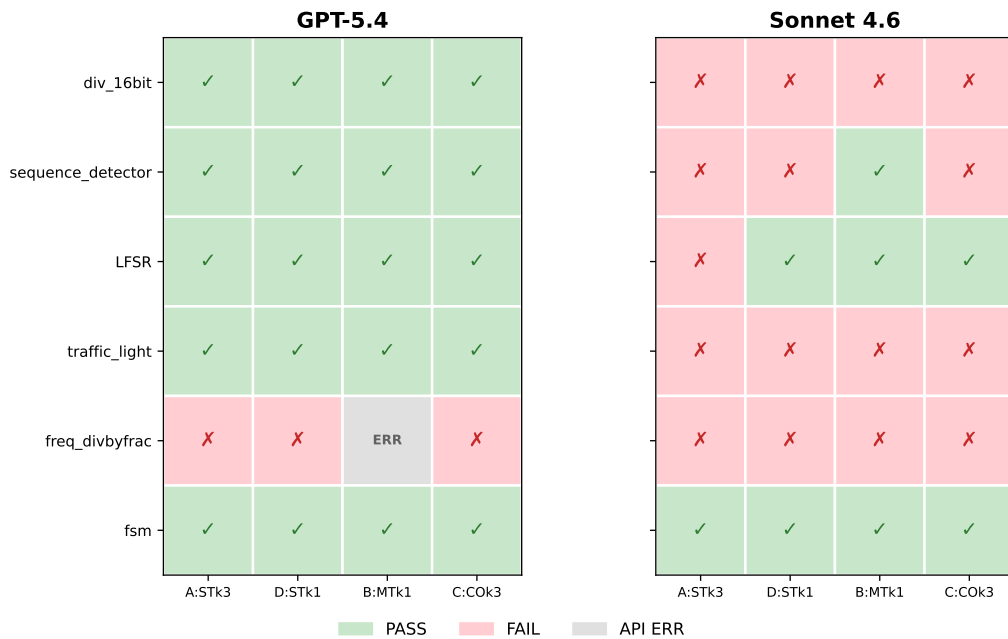


图 B.4 多轮对话反馈 v2 逐题矩阵

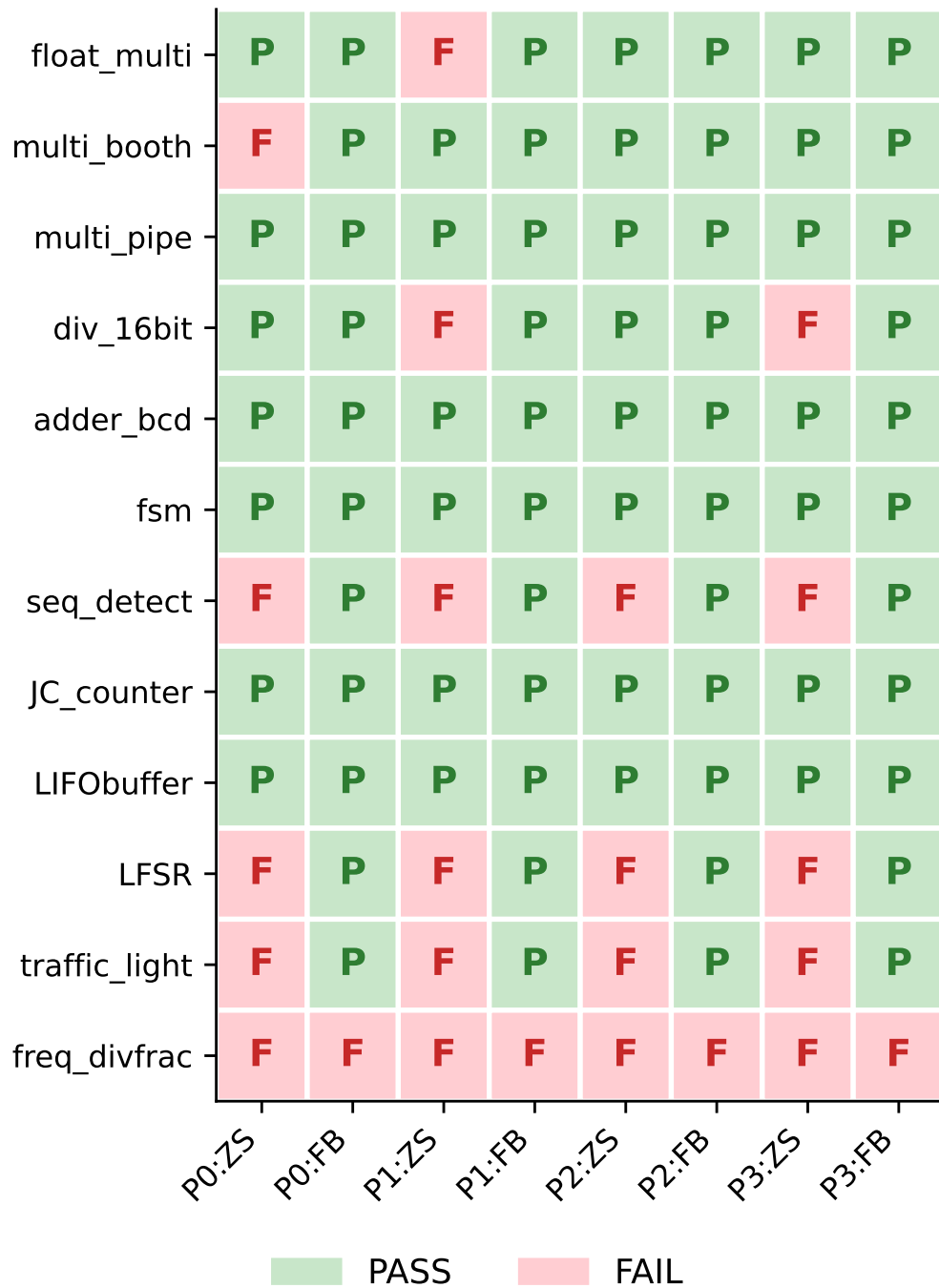


图 B.5 提示策略逐题结果矩阵